

Streaming Algorithms for Monotonicity Testing

Amir Azarmehr* Soheil Behnezhad* Lily Chung† Alma Ghafari* Jane Lange†
Ronitt Rubinfeld†

Abstract

Consider a poset—or equivalently an n -vertex DAG $G = (V, \vec{E})$ —and a boolean function $f : V \rightarrow \{0, 1\}$ on its vertex set. We say f is *monotone* if $f(u) \leq f(v)$ for all $(u, v) \in \vec{E}$. While there is extensive literature on the *query complexity* of testing monotonicity, we focus instead on the *space complexity* and initiate the study of this problem in the *streaming* setting. Namely, the edges of G arrive in an arbitrary order, and the goal is to estimate distance to monotonicity of a given function f using $\tilde{O}(n)$ space. Note that while this space allows receiving and storing f , it is much smaller than the input graph G which could have up to $\Omega(n^2)$ edges.

Our main result is an algorithm that $(1 + \varepsilon)$ -approximates distance to monotonicity in $\sqrt{n}^{1+o(1)}$ passes. We also prove that this is the best pass-complexity one can hope for, for any $O(1)$ -approximation, short of improving the state-of-the-art streaming algorithm for *st*-reachability, which is a very well-studied problem.

On the technical side, our algorithm approximates the size of maximum matching in (a subgraph of) the transitive closure of G . While the maximum matching problem has received significant attention in the streaming setting, the fact that we are computing it in the transitive closure requires very different ideas. In fact, a main contribution of our work is to connect sublinear time algorithms for estimating the maximum matching size to the streaming setting for the first time. While existing off-the-shelf sublinear time algorithms only result in an $n\sqrt{n}^{1+o(1)}$ pass algorithm in our setting, we show how to significantly improve upon them by allowing stronger queries (such as vertex and subset queries) that can be implemented just as efficiently as more standard adjacency matrix and list queries for our problem.

*Northeastern University, {azarmehr.a, s.behnezhad, ghafari.m}@northeastern.edu. A.A., S.B., and A.G. were supported in part by NSF CAREER Award CCF-2442812, a Sloan Fellowship, and a Google Research Award.

†MIT CSAIL, {lkdc, jlange}@mit.edu, ronitt@csail.mit.edu. L.C. and J.L. were supported by the Big George fellowship. L.C., J.L., and R.R. were supported by NSF TRIPODS Award DMS-2022448 and NSF Award CCF-2310818.

Contents

1	Introduction	1
1.1	Main Technique: Query Complexity of Maximum Matching Size	1
1.2	Future Directions and Open Problems	5
1.3	Further Related Work	5
2	Preliminaries	6
2.1	Definitions and Notation	6
2.2	Standard Facts	6
3	Global Matching Estimation	6
3.1	Validity	8
3.2	Approximation Ratio	11
4	Implementation With Subset Queries	12
4.1	Degree Estimation	12
4.1.1	Degree Estimation Through Geometric Sampling	13
4.1.2	Proof of Lemma 4.1	14
4.2	Low-Degree Phase Implementation	14
4.3	Proof of Theorem 4	19
5	Streaming Distance-to-Monotonicity Estimation	21
6	Implementation With Vertex Queries	23
6.1	Correctness and Approximation Ratio of the Modified Global Algorithm	24
6.2	Implementing the Modified Algorithm Using Vertex Queries	25
6.3	Proof of Theorem 2	26
7	Better Approximation Ratios with Subset Queries	27
8	The Lower Bound	31

1 Introduction

Consider a poset—or equivalently an n -vertex directed acyclic graph (DAG) $G = (V, \vec{E})$. Suppose we are also given a boolean function $f : V \rightarrow \{0, 1\}$ defined on the vertex set of this graph. Function f is said to be *monotone* if $f(u) \leq f(v)$ for all $(u, v) \in \vec{E}$. We study algorithms that efficiently determine the *distance to monotonicity* of f , denoted $d_{\text{mon}}(f)$, which is defined as the minimum number of entries of f that have to change in order to make f monotone.

Monotonicity testing has been studied extensively in the literature (see [Section 1.3](#)). However, nearly all these works study the *query complexity* of this problem. That is, they assume that the graph G is entirely known in advance and can be accessed for free, and study how many queries to f are needed in order to estimate its distance to monotonicity. Our focus instead is on massive inputs, where the graph G cannot be stored and randomly accessed by the algorithm.

More specifically, we study *streaming* algorithms for approximating distance to monotonicity: The edges of G arrive one by one in a stream, and the algorithm has space much smaller than this input. As standard in the literature of streaming graph algorithms, we allow $\tilde{O}(n)$ space¹, which is enough to receive and store the function f entirely, but is much smaller than the graph G which could include $\Omega(n^2)$ edges.

Our main result in this paper is the following algorithm:

Theorem 1. *For any $\varepsilon > 0$, there is a randomized streaming algorithm that takes $\sqrt{n}^{1+o(1)}$. $\text{poly}(1/\varepsilon)$ passes over the stream, uses $\tilde{O}(n/\varepsilon^2)$ space, and w.h.p. $(1 + \varepsilon)$ -approximates $d_{\text{mon}}(f)$.*

At first glance, the $\sqrt{n}^{1+o(1)}$ pass-complexity might appear prohibitive. However, we prove that this is the best bound one can hope for short of a breakthrough in streaming algorithms. Specifically, we show in [Theorem 9](#) that any p -pass $O(1)$ -approximation of $d_{\text{mon}}(f)$ also solves *st*-reachability in p passes and $\tilde{O}(n)$ space. Since the fastest known streaming *st*-reachability algorithm uses $\sqrt{n}^{1+o(1)}$ passes [[JLS19](#), [AJJ⁺22](#)], this shows that any faster algorithm for approximately determining the distance to monotonicity would also improve the pass-complexity of streaming *st*-reachability.

In fact, our algorithm uses the streaming version of [[JLS19](#)] directly to compute a *shortcut set*, a set of edges which can be added to the graph to reduce its diameter without changing the reachability relation on vertices. Our algorithm’s pass complexity is (ignoring $\text{poly}(\log n)$ factors) equivalent to the diameter of this shortcut set. Therefore, as long as the fastest known streaming *st*-reachability algorithms continue to work by computing a shortcut set, the pass-complexity of [Theorem 1](#) will match those algorithms.

1.1 Main Technique: Query Complexity of Maximum Matching Size

The key contribution of our work is to tie the streaming complexity of monotonicity testing to *sublinear time* algorithms for estimating the size of maximum matching. The latter problem has been studied extensively on its own over the last two decades, and has a rich literature with numerous applications (see [Section 1.3](#)). However, to our knowledge this is the first application of this problem in the streaming setting. Additionally, as we soon discuss, our work motivates the study of the query complexity of the maximum matching problem in alternative models, where the queries differ from

¹Here and throughout the paper we use $\tilde{O}(f)$ to suppress $\text{poly}(\log n)$ factors.

the standard adjacency-list or adjacency-matrix queries.

Background: Monotonicity and matchings. The connection between the maximum matching problem and monotonicity of boolean functions dates back to the early monotonicity testing work of [DGL⁺99]. Let G_f be the *violation graph* defined on the same vertex set V , which has an edge $\{u, v\}$ whenever u can reach v but $f(u) > f(v)$. Denoting the size of a maximum matching of G_f by $\mu(G_f)$, it is well-known that

$$d_{\text{mon}}(f) = \mu(G_f).$$

Therefore, all we need to do is to estimate the size of maximum matching in G_f . While the maximum matching problem is extremely well-studied in the streaming setting, the main difference here is that we only receive the graph G and function f in the stream which define G_f implicitly. Indeed, even for checking the existence of a single edge (u, v) in G_f one needs to solve the *st*-reachability problem in G , which itself takes $\sqrt{n}^{1+o(1)}$ passes.

Query complexity of maximum matching size. Since direct access to the graph G_f is expensive in our setting, we would like to estimate its maximum matching size without *querying* too many edges of G_f . Luckily, this is also a very well-studied problem. For example, an algorithm of [Beh21] obtains a $(2, \varepsilon n)$ -approximation of the maximum matching size by making $\tilde{O}(n)$ adjacency-matrix queries to the graph. That is, each such query specifies two vertices and the response is whether they are adjacent. Each such query to G_f can easily be answered in our setting by two calls to an *st*-reachability algorithm of [JLS19], hence the final algorithm will take

$$\underbrace{\sqrt{n}^{1+o(1)}}_{\text{cost of a pair query}} \cdot \underbrace{\tilde{O}(n)}_{\text{number of pair queries}} = n^{3/2+o(1)}$$

passes in total. Unfortunately, this significantly exceeds our desired $\sqrt{n}^{1+o(1)}$ pass-complexity. But it is known (see [Beh21, PR07]) that $\Omega(n)$ adjacency-matrix queries are indeed needed to obtain any constant approximation of maximum matching size, so this black-box reduction cannot possibly result in our desired bound.²

Our general idea for breaking through the $n\sqrt{n}$ barrier discussed above is to allow the matching size estimator to make *stronger* queries to the graph that can be implemented just as efficiently as adjacency-matrix queries in our model.

The vertex-query model. Our first observation is that the algorithm of [JLS19] is actually not specific to *st*-reachability, but rather solves the more general single source reachability problem in $\sqrt{n}^{1+o(1)}$ passes. For us, this implies that for a vertex v , we can gather the entire neighbor set $N_{G_f}(v)$ of v in just $\sqrt{n}^{1+o(1)}$ passes. This motivates the study of the query complexity of maximum matching in the *vertex query*³ model:

Definition 1.1 (Vertex queries). *In the vertex-query access model, the algorithm queries a vertex $v \in V$, and receives its neighborhood $N(v)$.*

Indeed, we show that this stronger query access model allows us to break the $\Omega(n)$ query lower bound for estimating the maximum matching size in the more standard query models. Namely, we prove that:

²This lower bound holds even in the more powerful access model where adjacency-matrix, adjacency-list index (i -th neighbor of v), and degree queries can be mixed adaptively.

³Some works in the literature of sublinear time algorithm also refer to such queries as *all-neighbor queries*.

Theorem 2. *Given an n -vertex graph G , there is a randomized algorithm that $(3, \varepsilon n)$ -approximates the size of maximum matching of G for any $\varepsilon > 0$ using $\sqrt{n} \cdot \text{poly}(\log n, 1/\varepsilon)$ vertex queries to G w.h.p.*

Plugging [Theorem 2](#) in the final algorithm instead of the pair query algorithm of [\[Beh21\]](#) improves the number of passes to

$$\underbrace{\sqrt{n}^{1+o(1)}}_{\text{cost of a vertex query}} \cdot \underbrace{\tilde{O}(\sqrt{n})}_{\text{number of vertex queries}} = n^{1+o(1)},$$

which is much fewer than $n^{3/2+o(1)}$ but still far from our desired $\sqrt{n}^{1+o(1)}$ bound.

The subset-query model. Our final observation is that the algorithm of [\[JLS19\]](#) can in fact be used to solve a *subset query* in just $\sqrt{n}^{1+o(1)}$ passes. Such queries are more powerful than vertex queries, and are defined as follows:

Definition 1.2 (Subset Queries). *In the subset-query access model, the algorithm queries a subset $S \subseteq V$, and receives its neighborhood $N(S) = \{u \in V : \exists v \in S \text{ such that } \{u, v\} \in E\}$.*

For us, it is crucial that each such query only returns the set $N(S)$ and *not* which vertices of S each vertex in $N(S)$ is adjacent to, as otherwise the algorithm of [\[JLS19\]](#) would not be applicable.

We show that subset queries allow for a more dramatic improvement in the query-complexity: only polylogarithmically many subset queries are required to estimate the matching size. The following theorem is our main technical contribution in this work:

Theorem 3. *Given an n -vertex graph G , for any $\varepsilon > 0$, there is a randomized algorithm that $(1.5 + \varepsilon)$ -approximates the size of maximum matching in G using $\text{poly}(\log n, 1/\varepsilon)$ subset queries to G w.h.p. If the graph is bipartite, then the approximation ratio improves to $1 + \varepsilon$.*

Now when we plug [Theorem 3](#) in the final algorithm, the number of passes will be

$$\underbrace{\sqrt{n}^{1+o(1)}}_{\text{cost of a subset query}} \cdot \underbrace{\text{poly}(\log n)}_{\text{number of subset queries}} = \sqrt{n}^{1+o(1)},$$

which immediately implies [Theorem 1](#). We note that we apply [Theorem 3](#) in a white-box manner in our final algorithm for [Theorem 1](#).

We also give a self contained proof for $(2 + \varepsilon)$ -approximation of maximum matching.

Theorem 4. *Given an n -vertex graph G , for any $\varepsilon > 0$, there is a randomized algorithm that $(2 + \varepsilon)$ -approximates the size of maximum matching in G using $\text{poly}(\log n) \cdot (1/\varepsilon)^{O(1/\varepsilon)}$ subset queries to G w.h.p.*

Approximating maximum matching size using vertex queries. We describe the high-level approach to proving [Theorem 4](#), which shows how to approximate the size of the maximum matching in G using subset queries.

Our starting point is a “peeling”-type algorithm [\[PR07, OR10, KKS14, CLM⁺20, KMNFT20\]](#) which iteratively matches and removes highest degree vertices. While we can’t implement this

algorithm in our setting, it is useful to first describe it and discuss what goes wrong in implementing it. In an idealized peeling algorithm, we divide the vertices into two halves randomly. Then each vertex from the first half “proposes” to a single neighbor chosen uniformly at random and independently. Then every vertex in the second half that receives at least one proposal “accepts” one of those arbitrarily and the two vertices get matched together. Now fix a vertex v and suppose that $\deg(v) \geq \Delta/2$ where Δ is the current maximum degree of the graph. Under the event that v is in the second half, it receives a proposal with probability at least $1 - (1 - 1/2\Delta)^{\Delta/2} \geq 0.1$. This means that after this process, a constant fraction of vertices of degree at least $\Delta/2$ get removed from the graph. We can therefore afford to remove all vertices of degree at least $\Delta/2$ from the graph regardless of whether they are matched, since a constant fraction of them have been matched in expectation. This reduces the maximum degree from Δ down to $\Delta/2$, so $O(\log \Delta)$ iterations of this process results in a constant approximate maximum matching in expectation.

As discussed, we cannot actually carry out this process in our setting. The main reason is that each vertex needs to propose to a randomly selected neighbor. Implementing this step requires $\Theta(n)$ neighbor queries, and subset queries are not helpful in reducing the query complexity to $\text{poly} \log(n)$. To get around this problem, we implement a different variant of the peeling algorithm that does not explicitly build a matching, but only provides an estimate of the matching size.

Our first ingredient is a degree estimation procedure in the subset query model, which we call a *subset degree estimator*. The subset degree estimator has the following functionality: Given a set of active vertices $U \subseteq V$, for every vertex $v \in V$ the procedure returns a $(1 + \varepsilon)$ -multiplicative approximation to $d_v := |N(v) \cap U|$. The procedure uses $O\left(\frac{\log^2 n}{\varepsilon^3}\right)$ subset queries and succeeds with probability $1 - 1/\text{poly}(n)$. Its underlying technique is a sampling argument that uses geometrically decreasing sampling probabilities.

To see how the estimator works, fix a vertex v and let $d_v = |N(v) \cap U|$, noting that the value of d is unknown to the algorithm. For a sampling probability p , include every vertex of U independently with probability p to set S and query $N(S)$ using a subset query. The event that $v \notin N(S)$ happens when none of the d neighbors of v is sampled, which happens with probability $(1 - p)^{d_v}$. When $p \approx 1/2d_v$, this probability is bounded away from 0 and 1 by a constant, hence by enough repetitions and a Chernoff bound, one could estimate the value of $(1 - p)^{d_v}$, and thus d_v , multiplicatively. By trying out different values of p between $1/\text{poly}(n)$ and 1 geometrically, we ensure that for each vertex v , one of these trials achieves $p \approx 1/2d_v$. The advantage of this approach is that a single set of subset queries suffices to estimate $|N(v) \cap U|$ for all vertices v simultaneously.

We next modify the standard peeling algorithm so that it constructs an implicit fractional matching rather than an integral matching. Suppose the current degree threshold is $\Delta_{\max}$, and let U be the set of vertices that have not yet been removed. Let $H \subseteq U$ contain the vertices whose degree in the induced graph of U is approximately $\Delta_{\max}$. For every $u \in H$, we add a small amount of fractional weight, on the order of $\varepsilon/\Delta_{\max}$, to each edge from u to another vertex in U . Therefore, the total weight of a vertex (that is the sum of weights on its edges) $u \in H$ increases proportionally to its degree in $G[U]$, whereas the total weight of a vertex $v \in U \setminus H$ increases proportionally to the number of its neighbors in H . Vertices are removed from U once their saturation becomes close to 1. Since every vertex in H has degree comparable to $\Delta_{\max}$, its total weight increases by $\Omega(\varepsilon)$ in every iteration, and hence all vertices in H are removed after $O(1/\varepsilon)$ iterations. We can then decrease $\Delta_{\max}$ and repeat the process on the remaining set U .

The subset degree estimator allows us to implement this fractional peeling process without storing all the edges of the graph. At the beginning of an iteration, we estimate $d_U(v) = |N(v) \cap U|$

for all vertices to identify the high degree set H . We then run the estimator again with H as the input, obtaining estimates of $d_H(v) = |N(v) \cap H|$ for all vertices. The estimates $d_U(v)$ and $d_H(v)$ determine the increase in each vertex saturation, so the algorithm only needs to maintain the active set and one saturation value per vertex.

Running this fractional process until every vertex is removed would naturally approximate the maximum fractional matching value. Using only the general $3/2$ integrality gap bound for fractional matchings gives a $(3 + \varepsilon)$ -approximation to the maximum matching. To obtain a $(2 + \varepsilon)$ -approximation, we use a separate low degree phase. The low degree phase entails additional technicality that is described in [Section 4.2](#).

Finally, using our subset degree estimator within the MWU framework of [\[Liu24\]](#), we obtain a $(1 + \varepsilon)$ -approximation to the minimum fractional vertex cover using $\text{poly}(\log n, 1/\varepsilon)$ subset queries. This gives a $(1.5 + \varepsilon)$ -approximation to maximum matching in general graphs, and a $(1 + \varepsilon)$ -approximation for bipartite graphs. This is proved in [Section 7](#).

1.2 Future Directions and Open Problems

Although adjacency-matrix and adjacency-list queries are the standard models in the literature on sublinear-time and query algorithms, our work highlights the importance of studying two stronger models: vertex queries and subset queries. For maximum matching, we establish strong separations between these models and the traditional query models, for which $\Omega(n)$ queries are necessary. An interesting open question is whether similar separations arise for other natural problems.

1.3 Further Related Work

Monotonicity testing. The query complexity of testing whether a function is monotone, namely distinguishing monotone functions from those that are ε -far from monotone (in the Hamming distance), is one of the earliest property testing problems to be considered [\[EKK⁺00, GGL⁺00\]](#). The tolerant version of the problem, distinguishing functions that are ε -close to monotone, from those that are ε' -far from monotone is considered in the work that introduces tolerant testing [\[PRR06\]](#) (see also [\[ACCL07\]](#)), where it was demonstrated that tolerant testing and the estimating the distance to having the property are essentially equivalent tasks. Since then, a large body of work has considered algorithms and lower bounds for testing and tolerant testing of monotonicity over various domains and distance measures (examples include [\[DGL⁺99, FLN⁺02, Fis04, ACCL08, SS10, BGJ⁺10, BCGM12, BRY14, CS13, CS14, CST14, CDST15, CS16, CWX17, KMS18, BCS18, CS19, BB15, BB21, LRV22, BCS20, PRW22, BCS23, BKKM23, Jr.23, Jr.24, CDL⁺24, BKR24, CS25, CDH⁺25, FJ25, Yos26\]](#)).

Property testing vs. streaming algorithms. A connection between property testing and streaming has been explored in [\[MMPS17, PS18, CFPS20\]](#) where it is shown how to transform constant query property testers to the random-order single-pass streaming model with constant space. Note that the aforementioned lower bounds demonstrate that in most settings that have been considered, monotonicity testing requires nonconstant dependence on the domain size of the function.

Sublinear time algorithms for maximum matching. Sublinear-time matching algorithms have been a central topic of study and have enjoyed sustained interest in the past few decades

[PR07, NO08, YYI09, ORRR12, LRY17, KMNT20, Beh21, BRRS23, BRR23b, BKS23b, BRR23a, BRR24, ABRR25, MRT25]. Beyond addressing a fundamental problem in graph theory, they are deeply connected to many other settings and problems, including spanning trees, Steiner trees, and the traveling salesman problem [CKK20, CKT23b, CKT23a, BRRS24, MRTV25, MRTV26]. In addition, they have served as a key tool in dynamic algorithms for estimating the maximum matching size [Beh23, BKS23a, BKS24, ABR24], and more recently in [BG24, AKK25] for maintaining an approximate maximum matching in dynamic graphs, with an update time tied to the density of Ruzsa-Szemerédi graphs. This work serves as yet another important implication of such algorithms.

2 Preliminaries

2.1 Definitions and Notation

For a graph $G = (V, E)$, n will refer to $|V|$, and we will denote by $\mu(G)$ the size of a maximum matching in G . We say that an algorithm succeeds with high probability if for any constant $c > 0$, the algorithm can be made to succeed with probability at least $1 - n^{-c}$. We call an estimate $\hat{x}$ of a value x an α -approximation if it satisfies

$$x/\alpha \leq \hat{x} \leq x,$$

and an (α, β) -approximation if it satisfies

$$x/\alpha - \beta \leq \hat{x} \leq x.$$

Definition 2.1 (Edge and label stream). *Let $G = (V, E)$ be a directed acyclic graph and $f : V \rightarrow \{0, 1\}$ be a boolean function. In an edge and label stream of G and f , edges of G and example-label pairs $(v, f(v))$ are revealed in an arbitrary order.*

Definition 2.2 (Violation graph). *For a DAG $G = (V, E)$ and a boolean function $f : V \rightarrow \{0, 1\}$, the violation graph G_f is the undirected graph which has an edge $\{u, v\}$ if u can reach v but $f(u) > f(v)$ (or v can reach u but $f(v) > f(u)$).*

2.2 Standard Facts

Fact 2.3 (Distance to monotonicity and matching in G_f). *For a DAG $G = (V, E)$ and a boolean function $f : V \rightarrow \{0, 1\}$, we have $d_{\text{mon}}(f) = \mu(G_f)$.*

Fact 2.4 (Integrality gap bound for fractional matchings). *For every graph G , $\nu_f(G) \leq \frac{3}{2}\mu(G)$, where $\nu_f(G)$ denotes the maximum size of a fractional matching in G .*

3 Global Matching Estimation

In this section, we prove validity and approximation ratio of the matching value computed by a “global” algorithm, ESTIMATEMATCHINGSIZE (Algorithm 1). This algorithm accesses the graph G via black-box oracles COMPUTEDEGREES and MAXIMALMATCHING which provide the following functionality:

Assumption 3.1 (COMPUTEDEGREES). *There is a function $\text{COMPUTEDEGREES}(G, U, \varepsilon)$ that takes an arbitrary graph $G = (V, E)$, subset $U \subseteq V$, and approximation parameter $\varepsilon > 0$, and returns a vector $\vec{d}$ such that $d_v = (1 \pm \varepsilon)|N(v) \cap U|$ for all $v \in V$.*

Assumption 3.2 (MAXIMALMATCHING). *There is a function $\text{MAXIMALMATCHING}(G, U)$ that takes an arbitrary graph $G = (V, E)$ and subset $U \subseteq V$, and returns a maximal matching in $G[U]$.*

Only these functions interact with the edges of G (and thus require an access model to be implemented); the rest of the algorithm only maintains information about the active set U and the saturation vector $\vec{x}$. In Section 4, we will implement these oracles in the subset-query model, and in Section 6 we will discuss how to adapt these ideas to the vertex-query model.

Algorithm 1: ESTIMATEMATCHINGSIZE(G, ε)

Input: Graph $G = (V, E)$, approximation parameter $\varepsilon > 0$
Output: Constant-factor estimate of the maximum matching size in G

```

1  $U \leftarrow V$  // Set of active vertices; i.e. those that haven't been deleted
2  $\Delta_{\max} \leftarrow n$  // degree threshold
3  $\eta \leftarrow 2/\varepsilon^2$  // Transition threshold to low-degree phase
4  $\vec{x} := (x_v)_{v \in V} \leftarrow \vec{0}$  // vector of fractional matching weights assigned to vertices
   /*  $\vec{y} := (y_e)_{e \in E} \leftarrow \vec{0}$ ; this is a fractional matching we track in the comments to help with the
      analysis, but it is never explicitly maintained by the algorithm */
5 while  $\Delta_{\max} \geq \eta$  do
6    $U, \vec{x} \leftarrow \text{PEELHIGHDEGREE}(G, U, \vec{x}, \Delta_{\max}, \varepsilon)$ 
7    $\Delta_{\max} \leftarrow (1 - \varepsilon)\Delta_{\max}$ 
8    $\vec{x} \leftarrow \text{MATCHLOWDEGREE}(G, U, \vec{x}, \varepsilon)$ 
9 return  $\hat{\mu} := \frac{1-\varepsilon}{1+\varepsilon} \cdot \frac{1}{2} \sum_v x_v$ 

```

Algorithm 2: PEELHIGHDEGREE($G, U, \vec{x}, \Delta_{\max}, \varepsilon$)

Input: Graph $G = (V, E)$, active set U , saturation values $\vec{x}$, degree threshold $\Delta_{\max}$, approximation parameter $\varepsilon > 0$
Output: Updated active set and saturation values

```

1  $\vec{d}_U \leftarrow \text{COMPUTEDEGREES}(G, U, \varepsilon)$  // counts neighbors in  $U$ , see Assumption 3.1
2  $H \leftarrow \{v \in U : (d_U)_v \geq (1 - \varepsilon)\Delta_{\max}\}$ 
3 while  $H \neq \emptyset$  do
4    $\vec{d}_H \leftarrow \text{COMPUTEDEGREES}(G, H, \varepsilon)$ 
5    $x_v \leftarrow x_v + \frac{\varepsilon}{2} \cdot \frac{(d_U)_v}{\Delta_{\max}}$  for all  $v \in U$ 
6    $x_v \leftarrow x_v + \frac{\varepsilon}{2} \cdot \frac{(d_H)_v}{\Delta_{\max}}$  for all  $v \in U \setminus H$ 
   /*  $y_{uv} \leftarrow y_{uv} + \frac{\varepsilon}{2\Delta_{\max}}$  for each ordered pair  $(u, v) \in E(G[U])$  with  $u \in H$  */
7    $H \leftarrow H \setminus \{v \in H : x_v \geq 1 - 2\varepsilon\}$ 
8    $U \leftarrow U \setminus \{v \in U : x_v \geq 1 - 2\varepsilon\}$ 
9 return  $U, \vec{x}$ 

```

Algorithm 3: MATCHLOWDEGREE($G, U, \vec{x}, \varepsilon$)

Input: Graph $G = (V, E)$, active set U , saturation vector $\vec{x}$, approximation parameter $\varepsilon > 0$

Output: Updated saturation vector

```
1 for  $1/\varepsilon - 1$  iterations do
2    $M \leftarrow \text{MAXIMALMATCHING}(G, U)$  // see Assumption 3.2
3    $x_v \leftarrow x_v + \varepsilon$  for all  $v \in M$ 
4    $U \leftarrow U \setminus \{v \in U : x_v \geq 1 - 2\varepsilon\}$ 
5 return  $\vec{x}$  */
```

3.1 Validity

Here we will show that the value returned by ESTIMATEMATCHINGSIZE is at most $\mu(G)$. To do this, we will first show that the vertex weights $\vec{x}$ correspond to an implicit fractional matching $\vec{y}$ that is always valid. Then, we will show that this fractional matching has an integrality gap of at most $1/(1 - \varepsilon)$, which we account for by scaling down all the x_v values by $1 - \varepsilon$ at the end of the algorithm.

Lemma 3.3 (Invariants of PEELHIGHDEGREE). *Suppose PEELHIGHDEGREE (Algorithm 2) is called with parameters satisfying*

$$\varepsilon \leq \frac{1}{3} \quad \text{and} \quad \Delta_{\max} \geq (1 - \varepsilon) \max_{v \in G} |N(v) \cap U|$$

and such that the following conditions are satisfied:

1. *For every $v \in V$, $(1 - \varepsilon) \sum_{u \in N(v)} y_{uv} \leq x_v \leq (1 + \varepsilon) \sum_{u \in N(v)} y_{uv}$.*
2. *For every $v \in V$, $x_v \leq 1 - \varepsilon$.*
3. *For every $v \in U$, $x_v \leq 1 - 2\varepsilon$.*

Then these three conditions are maintained by PEELHIGHDEGREE.

Furthermore, when PEELHIGHDEGREE returns, all vertices in U have degree at most $\Delta_{\max}$.

Proof. First, we observe that all vertices of degree at least $\Delta_{\max}$ are in H ; this is because Assumption 3.1 ensures that all such vertices have $d_U(v) \geq (1 - \varepsilon)\Delta_{\max}$ and are thus placed in H . Since all vertices in H are removed from U during the execution of PEELHIGHDEGREE, this ensures that all vertices remaining in U afterwards have degree at most $\Delta_{\max}$.

Now we show the invariants. Assume the invariants hold at the beginning of the t -th execution of the loop body. For $v \in V \setminus U$, x_v and all y_{uv} incident to v are unchanged, so we will analyze only vertices that are in U at the start of the t -th execution. To show the first invariant, consider the following cases.

1. $v \in H$: We increase x_v by $\frac{\varepsilon}{2} \cdot \frac{(d_U)_v}{\Delta_{\max}}$ and y_{uv} by $\frac{\varepsilon}{2\Delta_{\max}}$ for all $u \in N(v) \cap U$. By Assumption 3.1, we have $(d_U)_v = (1 \pm \varepsilon)|N(v) \cap U|$. Thus, after the update to x_v , we have

$$(x_v)_{t+1} = (x_v)_t + \frac{\varepsilon}{2} \cdot \frac{(d_U)_v}{\Delta_{\max}}$$

$$\begin{aligned}
&\geq \left[(1 - \varepsilon) \sum_{u \in N(v) \cap U} (y_{uv})_t \right] + \frac{\varepsilon}{2} \cdot \frac{(1 - \varepsilon) |N(v) \cap U|}{\Delta_{\max}} \\
&= (1 - \varepsilon) \sum_{u \in N(v) \cap U} \left[(y_{uv})_t + \frac{\varepsilon}{2\Delta_{\max}} \right] \\
&= (1 - \varepsilon) \sum_{u \in N(v) \cap U} (y_{uv})_{t+1}.
\end{aligned}$$

A similar argument shows the upper bound of $(1 + \varepsilon) \sum_{u \in N(v)} y_{uv}$.

2. $v \in U \setminus H$: We increase x_v by $\frac{\varepsilon}{2} \cdot \frac{(d_H)_v}{\Delta_{\max}}$ and y_{uv} by $\frac{\varepsilon}{2} \cdot \frac{1}{\Delta_{\max}}$ for all $u \in N(v) \cap H$. We have:

$$\begin{aligned}
(x_v)_{t+1} &= (x_v)_t + \frac{\varepsilon}{2} \cdot \frac{(d_H)_v}{\Delta_{\max}} \\
&\geq \left[(1 - \varepsilon) \sum_{u \in N(v) \cap U} (y_{uv})_t \right] + \frac{\varepsilon}{2} \cdot \frac{(1 - \varepsilon) |N(v) \cap H|}{\Delta_{\max}} \\
&= \left[(1 - \varepsilon) \sum_{u \in N(v) \cap (U \setminus H)} (y_{uv})_t \right] + \left[(1 - \varepsilon) \sum_{u \in N(v) \cap H} (y_{uv})_t + \frac{\varepsilon}{2\Delta_{\max}} \right] \\
&= (1 - \varepsilon) \cdot \sum_{u \in N(v) \cap U} (y_{uv})_{t+1}.
\end{aligned}$$

A similar argument shows the upper bound of $(1 + \varepsilon) \sum_{u \in N(v)} y_{uv}$.

To show the other invariants, first observe that the third invariant follows from the fact that the vertices not satisfying $x_v \leq 1 - 2\varepsilon$ are removed from U at the end of the loop body. For the second, observe that x_v for $v \in U$ increases by $\frac{\varepsilon}{2} \cdot \frac{(d_U)_v}{\Delta_{\max}}$ or $\frac{\varepsilon}{2} \cdot \frac{(d_H)_v}{\Delta_{\max}}$ in each iteration of the loop. By [Assumption 3.1](#) and the assumption that $\Delta_{\max} \geq (1 - \varepsilon) \max_{v \in G} |N(v) \cap U|$, we have

$$\frac{(d_U)_v}{\Delta_{\max}} \leq \frac{1 + \varepsilon}{1 - \varepsilon} \leq 2,$$

and similarly for $\frac{(d_H)_v}{\Delta_{\max}}$. Thus the value of x_v increases by at most ε in each iteration. Since $(x_v)_t \leq 1 - 2\varepsilon$ at the start of the iteration, $(x_v)_{t+1} \leq 1 - \varepsilon$. $\square$

We now state and prove a similar statement for MATCHLOWDEGREE.

Lemma 3.4 (Invariants of MATCHLOWDEGREE). *Suppose MATCHLOWDEGREE ([Algorithm 3](#)) is called under the conditions:*

1. For every $v \in V$, $(1 - \varepsilon) \sum_{u \in N(v)} y_{uv} \leq x_v \leq (1 + \varepsilon) \sum_{u \in N(v)} y_{uv}$.
2. For every $v \in V$, $x_v \leq 1 - \varepsilon$.
3. For every $v \in U$, $x_v \leq 1 - 2\varepsilon$.

Then these three conditions are maintained by MATCHLOWDEGREE.

Proof. Assume the invariants hold at the start of iteration t . As before, the third invariant follows from the fact that vertices not satisfying $x_v \leq 1 - 2\varepsilon$ are removed from U at the end of the iteration, and the second invariant follows from the third invariant and the fact that x_v increases by at most ε in each iteration.

To show the first invariant, consider a vertex v that is matched in M during the t -th iteration, since otherwise x_v and any y_{uv} incident to v would be unchanged. Let w be v 's matching partner and note that all y_{uv} for $u \neq w$ are also unchanged. We have then:

$$\begin{aligned} (x_v)_{t+1} &= (x_v)_t + \varepsilon \geq \left[(1 - \varepsilon) \sum_{u \in N(v)} (y_{uv})_t \right] + \varepsilon \\ &\geq \left[(1 - \varepsilon) \sum_{u \in N(v), u \neq w} (y_{uv})_t \right] + (1 - \varepsilon) [(y_{vw})_t + \varepsilon] \\ &= (1 - \varepsilon) \sum_{u \in N(v)} (y_{uv})_{t+1}. \end{aligned}$$

A similar argument shows the upper bound of $(1 + \varepsilon) \sum_{u \in N(v)} y_{uv}$. $\square$

Now we move on to bound the integrality gap of the fractional matching $\vec{y}$. We will use the following well-known fact, which gives a sufficient condition for a fractional matching to be close to an integral matching:

Fact 3.5 (Small-set blossom inequalities). *Suppose $y : E \rightarrow [0, 1]$ is a fractional matching such that for every odd-size set $S \subset V$ with $|S| < \frac{1}{\beta}$, the blossom inequality*

$$\sum_{e \in E(G[S])} y_e \leq \frac{|S| - 1}{2}$$

holds. Then there exists an integral matching M such that $|M| \geq (1 - \beta) \sum_{e \in E} y_e$.

This is because the fractional matching $(1 - \beta)y$ satisfies the blossom inequalities for all odd-size sets, which implies it is a convex combination of integral matchings [Edm65]. We will show that our fractional matching satisfies this condition with $\beta = \varepsilon$:

Lemma 3.6. *Assume $\varepsilon \leq \frac{1}{3}$, and let $\vec{x}$ be the vector computed by ESTIMATEMATCHINGSIZE (Algorithm 1). Then there exists a fractional matching in G with vertex saturation vector $\frac{1}{1+\varepsilon}\vec{x}$ that satisfies the blossom inequalities for all odd sets S of size at most $1/\varepsilon$.*

Proof. It follows from Lemmas 3.3 and 3.4 that the invariants on $\vec{x}$ and $\vec{y}$ are preserved throughout the execution of ESTIMATEMATCHINGSIZE. Therefore the final values of $\vec{x}$ and $\vec{y}$ satisfy $\frac{1}{1+\varepsilon}x_v \leq \sum_{u \in N(v)} y_{uv} \leq 1$ for every v ; thus $\vec{y}$ is a fractional matching with vertex saturations at least $\frac{1}{1+\varepsilon}\vec{x}$.

Now we claim the $\vec{y}$ values satisfy the small-set blossom inequalities.

Let L denote the set of vertices in U when MATCHLOWDEGREE is called. First we will bound the weight on edges with an endpoint outside of L . Observe that in PEELHIGHDEGREE, since $\frac{(d_U)_v}{\Delta_{\max}} \geq 1 - \varepsilon \geq 1/2$, the value of x_v for $v \in H$ increases by at least $\varepsilon/4$ each iteration; thus the number of iterations is at most $4/\varepsilon$. Each edge incident to H gains weight $\frac{\varepsilon}{2\Delta_{\max}}$ in each iteration,

so after $4/\varepsilon$ iterations, each edge will have gained weight at most $2/\Delta_{\max} \leq 2/\eta = \varepsilon^2$. Only edges incident to H gain weight, and each vertex appears in H at most once; thus, all edges with an endpoint outside of L satisfy $y_{uv} \leq \varepsilon^2$.

Consider a set $S \subseteq V$ of odd size $k \in [3, 1/\varepsilon]$, with ℓ vertices in L and $k - \ell$ vertices outside of L . We will bound the matching weight in $G[S]$. The contribution from edges with an endpoint outside of L is at most $\varepsilon^2 \cdot k(k - \ell) \leq \varepsilon(k - \ell)$ by the above argument. For the edges between members of L , all their weight comes from `MATCHLOWDEGREE`. Since there are $1/\varepsilon - 1$ iterations and each contributes ε weight to an integral matching, the contribution from L is at most $(1 - \varepsilon) \lfloor \frac{\ell}{2} \rfloor$; thus the inequality is immediately satisfied if $\ell = k$.

If $\ell \leq k - 1$, then the total weight is at most

$$\begin{aligned} (1 - \varepsilon) \frac{\ell}{2} + \varepsilon(k - \ell) &\leq (1 - \varepsilon) \frac{k - 1}{2} + \varepsilon \cdot 1 \\ &\leq \frac{k - 1}{2}, \end{aligned}$$

where the first inequality is valid because $\frac{1 - \varepsilon}{2} \geq \varepsilon$, and the second is because $k \geq 3$. $\square$

Corollary 3.7 (Validity of matching). *Let $\vec{x}$ be the saturation vector returned by [Algorithm 1](#) with $\varepsilon \leq 1/3$ and $\eta \geq 2/\varepsilon^2$. Then there exists an integral matching in G of value at least $\frac{1 - \varepsilon}{1 + \varepsilon} \cdot \frac{1}{2} \sum x_v$.*

Proof. By [Lemma 3.6](#), there is a fractional matching satisfying the small-set blossom inequalities with vertex saturations at least $\frac{1}{1 + \varepsilon} \vec{x}$. This matching has value at least $\frac{1}{1 + \varepsilon} \cdot \frac{1}{2} \sum x_v$. By [Fact 3.5](#), there is thus an integral matching in G of value at least $\frac{1 - \varepsilon}{1 + \varepsilon} \cdot \frac{1}{2} \sum x_v$. $\square$

3.2 Approximation Ratio

We have shown that the value returned by [Algorithm 1](#) is at most $\mu(G)$; i.e. it does not overestimate the matching size. In this subsection, we will show that `ESTIMATEMATCHINGSIZE` achieves an approximation ratio of $2 + O(\varepsilon)$.

Lemma 3.8. *The fractional matching value $\hat{\mu}$ returned by [Algorithm 1](#) satisfies*

$$\hat{\mu} \geq \frac{\mu(G)}{2 + O(\varepsilon)}.$$

To prove this, we will make use of the following duality relationship between vertex covers and matchings:

Fact 3.9. *Let x_v be a real vector over the vertices of a graph G . If $x_u + x_v \geq c$ for every edge $\{u, v\} \in E$, then $\sum_{v \in V} x_v \geq c\mu(G)$.*

Proof. Consider a maximum integral matching M^* in G and let $V(M^*)$ denote the set of its endpoints. Then

$$\sum_{v \in V} x_v \geq \sum_{\{u, v\} \in M^*} x_u + x_v \geq c\mu(G). \quad \square$$

Using this we can proceed with the bound on the approximation ratio of our algorithm.

Proof of Lemma 3.8. We will prove that every edge $(u, v) \in G$ satisfies $x_u + x_v \geq 1 - 2\varepsilon$. Let L be the set of vertices in U when MATCHLOWDEGREE is called. Consider the following 2 cases:

1. $u \in V \setminus L$ or $v \in V \setminus L$: In this case, one of u or v is removed by the subroutine PEELHIGHDEGREE (Algorithm 2). Since vertices are only removed under the condition $x_v \geq 1 - 2\varepsilon$, we have $x_u + x_v \geq 1 - 2\varepsilon$.
2. $u, v \in L$: Either u or v is removed due to its value exceeding $1 - 2\varepsilon$, or the edge (u, v) persists through the entirety of MATCHLOWDEGREE. In the latter case, since every matching M is maximal, either u or v must be matched in M . Thus in each iteration, ε is added to the value of x_u or x_v . Since there are $1/\varepsilon - 1$ iterations, we have $x_u + x_v \geq 1 - \varepsilon$.

We have shown that in both cases, $x_u + x_v \geq 1 - 2\varepsilon$; thus Fact 3.9 shows that $\sum_{x \in V} x_v \geq (1 - 2\varepsilon)\mu(G)$, which concludes the proof. $\square$

4 Implementation With Subset Queries

We have given an algorithm that estimates matching size, given an oracle that provides degree estimates and an oracle that provides maximal (integral) matchings. In this section, we will modify this algorithm to work with subset-query access to the graph (Definition 1.2). In Section 4.1 will straightforwardly implement the COMPUTEDEGREES oracle with subset queries.

Then, in Section 4.2, we will present a query-efficient alternative to Algorithm 3: instead of computing global matchings, we give a *local computation algorithm (LCA)* that computes the output value x_v at a single vertex v by exploring (in expectation) $\Delta^{O(1/\varepsilon)}$ vertices near v . The algorithm works by recursively running the LCA for randomized greedy maximal matching, with each level of recursion representing an iteration of Algorithm 3. This allows us to determine how many times v is matched and thus how much to increase x_v . We use this to sample from the output vector $\vec{x}$ in order to estimate the final matching value.

4.1 Degree Estimation

In this section, we will implement the functionality of COMPUTEDEGREES as specified in Assumption 3.1, in the subset-query model. The goal is to prove the following:

Lemma 4.1 (Estimating degrees with subset queries). *There is an algorithm that uses $O\left(\frac{\log^2 n}{\varepsilon^3}\right)$ subset queries and $O(n \log(n/\varepsilon))$ space, implements the functionality of Assumption 3.1 when it succeeds, and succeeds with high probability.*

To simplify the proof, we abstract away the details concerning the graph as follows. The goal is to estimate a hidden value d representing the (induced) degree of a vertex; i.e. the number of neighbors it has in some subset. We are allowed to learn about d by sampling independently from the distribution family $\left\{ \text{Bernoulli}_{(1-p)^d} \right\}_{p \in [0,1]}$. This is equivalent to sampling every vertex with a probability p , and checking if v is a neighbor of any of them, which happens with probability $1 - (1 - p)^d$. We prove that such Bernoulli samples, represented by $Y_i^{(r)}$ in Algorithm 4, can be used to estimate the value of d up to a multiplicative factor of $(1 + \varepsilon)$.

4.1.1 Degree Estimation Through Geometric Sampling

Algorithm 4: Degree Estimation Through Geometric Sampling

Input: Domain size n , accuracy parameters ε and δ , access to independent samples from distributions $\mathcal{D}_p := \text{Bernoulli}((1-p)^d)$ for $p \in [0, 1]$, where $d \leq n$ is the hidden value

Output: An estimate for d

```

1 Let  $L \leftarrow \lceil \log_{1+\varepsilon} 4n \rceil$ ,  $R \leftarrow 500 \frac{\log L + \log(1/\delta)}{\varepsilon^2}$ , and  $p_i \leftarrow (1+\varepsilon)^{-i}$ 
2 for  $i \in [L]$  and  $r \in [R]$  do
3   Draw  $Y_i^{(r)} \sim \mathcal{D}_{p_i}$  independently
4    $Z_i \leftarrow \frac{1}{R} \sum_{r=1}^R Y_i^{(r)}$ 
5  $i^* \leftarrow \min\{i \mid Z_i \geq 1/2\}$ 
6 return  $\hat{d} := \frac{\ln Z_{i^*}}{\ln(1-p_{i^*})}$ 

```

Lemma 4.2. *The output of Algorithm 4 satisfies $\hat{d} = (1 \pm \varepsilon)d$, with probability $1 - \delta$.*

To provide some intuition, we note that since each $Y_i^{(r)}$ is a Bernoulli random variable with mean $q_i := (1-p_i)^d$, Z_i provides an estimate of q_i . Furthermore, for any $i \in [L]$, it holds that $\frac{\ln q_i}{\ln(1-p_i)} = d$. The choice of i^* is such that $q_{i^*} = \Theta(1)$ and equivalently $p_{i^*} = \Theta(\frac{1}{d})$. As a result, $\frac{\ln Z_{i^*}}{\ln(1-p_{i^*})}$ provides an accurate estimate of d . We formalize the proof below. The analysis uses an additional parameter, $\lambda = \varepsilon/20$.

Claim 4.3. *It holds that $|Z_i - q_i| \leq \lambda$ for all i , with probability $1 - \delta$.*

Proof. This follows from a direct application of Hoeffding's inequality. Z_i is the average of R independent 0-1 variables, with an expected value of q_i . Therefore, for each i it holds:

$$\Pr(|Z_i - q_i| > \lambda) \leq 2e^{-2R\lambda^2} \leq \frac{\delta}{L}.$$

Hence, taking the union bound over $i \in [L]$ implies that with probability $1 - \delta$, it holds that $|Z_i - q_i| \leq \lambda$ for all i . $\square$

Claim 4.4. *Assuming the event in Claim 4.3 holds, we have that $q_{i^*} \in [1/4, 3/4]$.*

Proof. First, note that $q_{i^*} \geq \frac{1}{4}$ holds, since

$$q_{i^*} \geq Z_{i^*} - \lambda \geq \frac{1}{2} - \lambda \geq \frac{1}{4}.$$

To upper-bound q_{i^*} , let $j = \min\{j \mid p_j \leq \frac{1}{3d}\}$. That is, $\frac{1}{3(1+\varepsilon)} \leq d \cdot p_j \leq \frac{1}{3}$. As a result, we have

$$q_j = (1-p_j)^d \geq 1 - d \cdot p_j \geq \frac{2}{3}.$$

Additionally, we have that $Z_j \geq \frac{2}{3} - \lambda \geq \frac{1}{2}$, and hence $i^* \leq j$. Therefore, it holds

$$q_{i^*} \leq q_j = (1-p_j)^d \leq e^{-d \cdot p_j} \leq e^{-1/3(1+\varepsilon)} \leq \frac{3}{4},$$

which concludes the proof. $\square$

Claim 4.5. *It holds that $\left| \frac{\ln Z_{i^*}}{\ln(1-p_{i^*})} - d \right| \leq \varepsilon d$, with probability $1 - \delta$.*

Proof. Recall that $\frac{\ln q_{i^*}}{\ln(1-p_{i^*})} = d$, and with probability $1 - \delta$, we have $|Z_{i^*} - q_{i^*}| \leq \lambda$ and $q_{i^*} \in [1/4, 3/4]$. Together, these imply

$$\begin{aligned} \left| \frac{\ln Z_{i^*}}{\ln(1-p_{i^*})} - d \right| &= \left| \frac{\ln Z_{i^*} - \ln q_{i^*}}{\ln(1-p_{i^*})} \right| \\ &\leq \frac{4\lambda}{|\ln(1-p_{i^*})|} \end{aligned} \tag{1}$$

$$\begin{aligned} &= \frac{4\lambda d}{|\ln(q_{i^*})|} \\ &\leq 16\lambda d \\ &\leq \varepsilon d. \end{aligned} \tag{2}$$

Here, the two equalities follow from $\frac{\ln q_{i^*}}{\ln(1-p_{i^*})} = d$, (1) follows from $Z_{i^*}, q_{i^*} \geq \frac{1}{4}$ and $|Z_{i^*} - q_{i^*}| \leq \lambda$, and (2) follows from $q_{i^*} \leq \frac{3}{4}$. This concludes the proof of the claim and [Lemma 4.2](#). $\square$

4.1.2 Proof of [Lemma 4.1](#)

We now implement geometric sampling with subset queries, with the degree vector as the hidden values, and we bound the query and space complexity.

Proof of [Lemma 4.1](#). First we will bound the space requirements of [Algorithm 4](#), which we will call with $\delta := n^{-(c+1)}$ for some constant c . The counters i, r , and Z_i can all be stored in $O(\log \log(n/\delta) + \log(1/\varepsilon))$ space, while $\hat{d}$ itself can require up to $\log((1+\varepsilon)n) = O(\log n)$ space. Thus we can bound the space by $O(\log(n/\varepsilon))$.

Observe that if [Algorithm 4](#) is run for each vertex v with hidden value $|N(v) \cap U|$ and failure probability $\delta := n^{-(c+1)}$, then it implements the behavior specified in [Assumption 3.1](#) with probability $\geq 1 - n^{-c}$ by a union bound over vertices. We will implement the sampling of $\text{Bernoulli}((1-p)^d)$ in parallel for all vertices by sampling each member of U independently with probability p . Let S be the set of sampled vertices and let T be the set returned by the subset query. We have for each v :

$$\Pr[v \notin T] = \prod_{u \in N(v) \cap U} u \notin S;$$

thus the vector $(\mathbb{1}[v \notin T])_{v \in V}$ has marginal distributions $\text{Bernoulli}((1-p)^{|N(v) \cap U|})$.

We run [Algorithm 4](#) in parallel for all vertices, so the space requirement becomes $O(n \log(n/\varepsilon))$. The number of samples required for each vertex is $LR = O\left(\frac{\log^2 n}{\varepsilon^2 \log(1+\varepsilon)}\right) = O\left(\frac{\log^2 n}{\varepsilon^3}\right)$, so this is the number of subset queries required. $\square$

4.2 Low-Degree Phase Implementation

This section is devoted to the implementation of the low-degree integral matching phase ([Algorithm 3](#)). Our implementation of this phase happens to use only vertex queries (which are a special case of subset queries), but we do not make use of this fact. We prove the following lemma.

Lemma 4.6. *Given a graph G with maximum degree Δ , active set $U \subseteq V(G)$, saturation vector $\vec{x}$, and an approximation parameter $\varepsilon > 0$, let v be a vertex chosen uniformly at random. There is an algorithm that computes the update saturation value of v as in [Algorithm 3](#) using $\Delta^{O(1/\varepsilon)}$ vertex queries in expectation, where the expectation is over the randomness of the algorithm and the choice of v .*

To implement the low-degree phase, we employ the scheme of [\[YYI09\]](#). Before describing the implementation, we review the definition of randomized greedy maximal matching (RGMM), which is used as a subroutine. Given a graph G , a maximal matching can be computed by iterating the edges in a random order π , and adding edges to the matching greedily (i.e., any iterated edge that has no adjacent edge in the matching is added to the matching). We refer to this matching as the greedy maximal matching corresponding to order π , $\text{GMM}(G, \pi)$, and the output of the entire process as the randomized greedy maximal matching.

[\[YYI09\]](#) analyze a recursive implementation of this algorithm that, given a specific edge e and an ordering π , determines whether e appears in $\text{GMM}(G, \pi)$. To do so, the algorithm recursively checks whether any adjacent edge e' that appears earlier in the order is in $\text{GMM}(G, \pi)$. If so, then e does not. Otherwise, e must be added to the maximal matching. The algorithm is formalized below ([Algorithm 5](#)) where, instead of a permutation, π is a mapping from the edges to $[0, 1]$. The key idea is that the neighboring edges can be checked in order of π , and as soon as any of them is added to the maximal matching, the rest need not be checked. Using this improvement, they show that the following holds.

Lemma 4.7 (follows from [\[YYI09, Lemma 2.3\]](#)). *Let $Q(G, \pi, e)$, the number of **in-queries** of e , be defined as the number of (recursive) calls made to $\text{GMM}(G, \pi, e)$ as a result of calling $\text{GMM}(G, \pi, e')$ once for every edge e' . Then, for **any** edge e , it holds that*

$$\mathbb{E}_{\pi}[Q(G, \pi, e)] \leq 1 + 2\Delta,$$

where Δ is the maximum degree of the graph, and the expectation is over the ordering π .

Algorithm 5: $\text{GMM}(G, \pi, e)$ [\[YYI09\]](#)

Input: Graph $G = (V, E)$, edge ranks $\pi : E \rightarrow [0, 1]$, and an edge e

Output: A boolean denoting whether e appears in the greedy maximal matching corresponding to π

```

1 for  $e'$  adjacent to  $e$ , with  $\pi_{e'} < \pi_e$ , in increasing order of  $\pi_{e'}$  do
2   if  $\text{GMM}(G, \pi, e')$  then
3     return FALSE
4 return TRUE
```

With the RGMM subroutine at hand, we move on to the implementation of [Algorithm 3](#), outlined as [Algorithms 6](#) and [7](#). For the maximal matching, in each of the $1/\varepsilon$ iterations, we use an RGMM on the active vertices. We implement this by adding another layer of recursion to [Algorithm 5](#), on the iteration number. More accurately, the algorithm consists of two recursive functions $\text{EDGEVALUE}(e, t)$ and $\text{VERTEXVALUE}(v, t)$ which compute the updated saturation values after t iterations, for an edge e or a vertex v , respectively. To compute the value of a vertex v after t iterations, we simply compute the value of the edges adjacent to it and add them to the

initial saturation value x_v . To compute the value of an edge after t iterations, we need to determine two things: (1) its value after $t - 1$ iterations, and (2) whether it appears in the t -th maximal matching (if it does, the value shall increase by ε). To assert whether an edge appears in the t -th maximal matching, we use the recursive algorithm for RGMM on the active vertices. Here, the active neighbors of a vertex are determined on the fly by recursively computing the vertex values after $t - 1$ iterations, and checking if they are at most $1 - 2\varepsilon$.

Algorithm 6: VERTEXVALUE($G, U, \vec{x}, v, t$)

Input: Graph $G = (V, E)$, active set U , saturation vector $\vec{x}$, vertex v , and the number of iterations t

Output: The updated saturation value of v , after t iterations of MATCHLOWDEGREE($G, U, \vec{x}$) (i.e., Algorithm 3)

```

1  $x'_v \leftarrow x_v$ 
2 for  $u \in N(v) \cap U$  do
3    $x'_v \leftarrow x'_v + \text{EDGEVALUE}(G, U, \vec{x}, (u, v), t)$ 
4 return  $x'_v$ 

```

The proof of correctness is straightforward. For the query complexity, note that the guarantee of Lemma 4.7 bounds the number of in-queries for any edge e by $O(\Delta)$, in expectation, when we call GMM(e') for a uniformly random edge e' . While this bounds the number of (out-)queries required for computing GMM(e') for an average edge e' , certain edges might have a much larger expected number of out-queries. Here, the number of out-queries of e' refers to the total number of recursive calls made as a result of invoking GMM(e'). The main challenge is that with the added recursion on the iteration t , a uniform query to the t -th level might result in non-uniform queries to the $(t - 1)$ -th level. That is, as a result of querying EDGEVALUE(e, t) for a uniformly random e , the queries to EDGEVALUE($\cdot, t - 1$) may be concentrated on an edge e' for which the number of expected number of out-queries is significantly larger than Δ . As a result, Lemma 4.7 does not yield a trivial $\Delta^{O(1/\varepsilon)}$ bound for the query complexity of EDGEVALUE. The key intuition here is that while the queries made to the lower levels may be non-uniform, they are not far from uniform. Loosely speaking, the distribution is distorted by a factor of $\text{poly}(\Delta)$ with each level of recursion, which leads to a $\Delta^{O(1/\varepsilon)}$ query complexity. The proofs are formalized below.

Proof of Lemma 4.6. For the sake of brevity, we elide the inputs G , U , and $\vec{x}$ of VERTEXVALUE and EDGEVALUE. We also note that EDGEVALUE is always called on edges that are initially active, i.e., they are in $G[U]$. As such, we only consider those edges in the analysis below.

Correctness. First, we prove correctness of these routines, i.e., that VERTEXVALUE(v, t) always correctly computes the updated saturation value, and EDGEVALUE(e, t) always correctly computes the updated value and whether e appears in the t -th selected maximal matching (here, correctness is defined with respect to sequentially selecting t RGMMs in the active graph, as in Algorithm 3). This is established by induction on t . For the base case $t = 0$, the claim holds trivially, as EDGEVALUE($e, 0$) returns $(0, \text{FALSE})$, and VERTEXVALUE($v, 0$) returns the initial value x_v .

For $t > 0$, note that the correctness of VERTEXVALUE(v, t) is implied by the correctness of EDGEVALUE(e, t), since VERTEXVALUE simply computes the updated values of the neighboring edges and adds them to the initial saturation x_v . Thus, it suffices to prove the correctness of EDGEVALUE(e, t). By the induction hypothesis, the recursive calls of EDGEVALUE(e, t) to

Algorithm 7: EDGEVALUE($G, U, \vec{x}, e, t$)

Input: Graph $G = V(V, E)$, active set U , saturation vector $\vec{x}$, edge $e \in G[U]$, and the number of iterations t

Output: The fractional value of e , after t iterations of MATCHLOWDEGREE($G, U, \vec{x}$) (i.e., Algorithm 3), and a boolean denoting whether e appears in the t -th maximal matching

```
1 if  $t = 0$  then                                     // base case
2   | return 0, FALSE
3 else
4   | /* Recursively recover the state of  $e$  after  $t-1$  iterations, and implement RGMM if the
5     | endpoints of  $e$  have not been saturated */
6   |  $u, v \leftarrow$  endpoints of  $e$ 
7   |  $x_e^{(t-1)} \leftarrow$  EDGEVALUE( $G, U, \vec{x}, e, t-1$ )
8   |  $x_u^{(t-1)} \leftarrow$  VERTEXVALUE( $G, U, \vec{x}, u, t-1$ )
9   |  $x_v^{(t-1)} \leftarrow$  VERTEXVALUE( $G, U, \vec{x}, v, t-1$ )
10  | if  $x_u^{(t-1)} \geq 1 - 2\varepsilon$  and  $x_v^{(t-1)} \geq 1 - 2\varepsilon$  then //  $e$  was deactivated in the previous iterations
11  |   | return  $x_e^{(t-1)}$ , FALSE
12  |  $N(e) \leftarrow \{e' \in G[U] \mid e' \text{ shares an endpoint with } e\}$  // query adjacency lists
13  | for  $e' \in N(e)$  do // assign ranks to the neighboring edges
14  |   | if  $e'$  has not already been assigned a rank then
15  |     | Draw  $\pi_{e'}^{(t)}$  uniformly from  $[0, 1]$ 
16  |   | for  $e' \in N(e)$ , such that  $\pi_{e'}^{(t)} < \pi_e^{(t)}$ , in increasing order of  $\pi_{e'}^{(t)}$  do
17  |     |  $x_{e'}^{(t)}, b \leftarrow$  EDGEVALUE( $G, U, \vec{x}, e', t$ )
18  |     | if  $b = \text{TRUE}$  then //  $e'$  appears in the GMM, hence  $e$  does not
19  |       | return  $x_e^{(t-1)}$ , FALSE
20  | return  $x_e^{(t-1)} + \varepsilon$ , TRUE // no lower-rank neighboring edges appear in the GMM, hence  $e$  does
```

EDGEVALUE($\cdot, t-1$) and VERTEXVALUE($\cdot, t-1$) yield the correct output. As a result, EDGEVALUE(v, t) can correctly determine the active subgraph after $t-1$ iterations, and compute the RGMM corresponding $\pi^{(t)}$. This concludes the proof of correctness.

Query Complexity. Moving on to the query complexity, we introduce some notation. Let $\pi^{(<t)}$ denote ranks used for computing the first $t-1$ RGMMs. For the sake of analysis, we can assume that all the ranks are drawn for the entire graph at the start of the algorithm. Consider the process of drawing all the ranks and invoking VERTEXVALUE($v, 1/\varepsilon - 1$) once for each vertex v . Let $Q(e', t)$, the in-degree of an edge e' at level t , be a random variable denoting the number of recursive calls made to EDGEVALUE(e', t) as the result of the aforementioned process.

We claim that it suffices to prove $\mathbb{E}[Q(e, 0)] = \Delta^{O(1/\varepsilon)}$ for all e . To see this, first note that the adjacency lists are accessed at most twice in each call to EDGEVALUE (once for each endpoint, to obtain all the neighboring edges), and once in each call to VERTEXVALUE (to obtain the adjacent edges). Therefore, it suffices to bound the total number of (recursive) calls to VERTEXVALUE and EDGEVALUE by $n\Delta^{O(1/\varepsilon)}$. Furthermore, except the initial calls to VERTEXVALUE($v, 1/\varepsilon - 1$), every call to VERTEXVALUE($\cdot, t$) is triggered by a call to EDGEVALUE($\cdot, t+1$), and each call to EDGEVALUE($\cdot, t+1$) is responsible for at most two of the calls to VERTEXVALUE($\cdot, t$). Therefore, it suffices to bound the total number of calls just to EDGEVALUE by $n\Delta^{O(1/\varepsilon)}$. Additionally, for $t > 0$, each call to EDGEVALUE(e, t) makes a call to EDGEVALUE($e, t-1$). Hence, since there are $1/\varepsilon$ levels, it suffices to bound the total number of calls made to EDGEVALUE($\cdot, 0$) by $n\Delta^{O(1/\varepsilon)}$. Finally, note that there are at most $m \leq n\Delta$ edges. As a result, it suffices to bound the number of calls to EDGEVALUE($e, 0$) by $\Delta^{O(1/\varepsilon)}$ for each edge e .

To prove $\mathbb{E}[Q(e, 0)] = \Delta^{O(1/\varepsilon)}$, we use a downward induction on $t = 1/\varepsilon - i$, and show

$$\mathbb{E}[Q(e, 1/\varepsilon - i)] \leq (4\Delta)^{2i}.$$

In fact, we prove a stronger claim that this holds even when we condition on an arbitrary set of ranks $\pi^{(<t)}$ (i.e., no matter what the graph is after the first $t-1$ iterations). The base case, $t = 1/\varepsilon - 1$, follows directly from Lemma 4.7. Conditioning on an arbitrary set of ranks $\pi^{(<t)}$, yields a set of active vertices U_t at the beginning of the t -th iteration. We are drawing a random rank $\pi^{(t)}$ for the t -th level, and invoking VERTEXVALUE(v, t) once for each v . These result in direct calls to EDGEVALUE(e, t) twice for each edge e . Then, EDGEVALUE(e, t) essentially simulate RGMM (Algorithm 5) on $G[U_t]$. Therefore, by Lemma 4.7, we can conclude:

$$\mathbb{E}[Q(e, t) \mid \pi^{(<t)}] \leq 2(1 + 2\Delta) \leq (4\Delta)^2,$$

for all e .

For the induction step, take $t = 1/\varepsilon - i$ for $i > 1$, fix a set of ranks $\pi^{(<t)}$, and assume that the hypothesis holds for $t+1$, i.e. for any $\pi^{(<t+1)}$, we have

$$\mathbb{E}[Q(e, t+1) \mid \pi^{(<t+1)}] \leq (4\Delta)^{2(i-1)},$$

for all e , where we are specifically interested in $\pi^{(<t+1)}$ that extend $\pi^{(<t)}$ by adding one level $\pi^{(t)}$. Lifting the condition on $\pi^{(t)}$, the above implies:

$$\mathbb{E}[Q(e, t+1) \mid \pi^{(<t)}] \leq (4\Delta)^{2(i-1)}.$$

We consider the calls initially made to level t as a result of the calls to level $t+1$. Each call to EDGEVALUE($e, t+1$) makes a call to EDGEVALUE(e, t) and two calls to VERTEXVALUE($\cdot, t$), one for

each endpoint. The calls to $\text{VERTEXVALUE}(\cdot, t)$, in turn, make a call to $\text{EDGEVALUE}(e', t)$ for each adjacent edge e' . We refer to these as the initial calls to level $\text{EDGEVALUE}(\cdot, t)$. Combining with the induction hypothesis, the number of initial calls made to $\text{EDGEVALUE}(e, t)$ can be bounded by $(1 + 2\Delta)(4\Delta)^{2(i-1)} \leq (4\Delta)^{2i-1}$ in expectation for any e .

The key step here is to artificially increase the expected number of calls to level t to reach $(4\Delta)^{2i-1}$.⁴ That is, if the expectation is lower for any edge e , we add extra queries to $\text{EDGEVALUE}(e, t)$. With each $\text{EDGEVALUE}(e, t)$ called exactly $(4\Delta)^{2i-1}$ times in expectation, we can invoke [Lemma 4.7](#) again to obtain:

$$\mathbb{E}[Q(e, t) \mid \pi^{(<t)}] \leq (4\Delta)^{2i-1} \cdot (1 + 2\Delta) \leq (4\Delta)^{2i}.$$

This concludes the proof of the induction and the claim. $\square$

4.3 Proof of [Theorem 4](#)

We will now combine the bounds of [Lemma 4.1](#) and [Lemma 4.6](#) to give a bound on the number of subset queries required to implement $\text{ESTIMATEMATCHINGSIZE}$ in its entirety. From this, we will conclude [Theorem 4](#).

First—primarily because the streaming algorithms will use a different approach for the low-degree phase—we will just bound the number of subset queries made during the high-degree phase of the algorithm, i.e. before MATCHLOWDEGREE is called.

Claim 4.8. *$\text{ESTIMATEMATCHINGSIZE}$, outside of MATCHLOWDEGREE , makes $\text{poly}(\log n, 1/\varepsilon)$ subset queries.*

Proof. Subset queries outside of MATCHLOWDEGREE are only made within COMPUTEDEGREES , so we will bound the number of calls to COMPUTEDEGREES . Each call to PEELHIGHDEGREE makes $O(1/\varepsilon)$ calls to COMPUTEDEGREES (see proof of [Lemma 3.6](#) for this argument), and there are $\leq \frac{\log n}{\log((1-\varepsilon)^{-1})}$ calls to PEELHIGHDEGREE . Thus we can bound the total number of calls to COMPUTEDEGREES by $O((\log n)/\varepsilon)$. By [Lemma 4.1](#), each call to COMPUTEDEGREES makes $O((\log(n/\delta) \log n)/\varepsilon^3)$ subset queries, where δ is the failure probability of COMPUTEDEGREES . For our algorithm to have a global failure probability of $1/n$, we will let $\delta := c \cdot \varepsilon/n \log n$, where c is determined by the hidden constant in the $O((\log n)/\varepsilon)$ bound on the number of calls to COMPUTEDEGREES .

This gives a final subset-query complexity of

$$O\left(\frac{\log n}{\varepsilon} \cdot \frac{\log(n/\varepsilon) \log n}{\varepsilon^3}\right) = \text{poly}(\log n, 1/\varepsilon). \quad \square$$

Now we will combine this with the bounds on the complexity of MATCHLOWDEGREE to conclude [Theorem 4](#). In fact we will show a slight relaxation of the theorem with additive-multiplicative approximation and expected query complexity; standard arguments show that the stronger version follows from that.

Theorem 5 (Relaxation of [Theorem 4](#)). *Given an n -vertex graph G , for any $\varepsilon > 0$, there is a randomized algorithm that $(2, \varepsilon n)$ -approximates the size of maximum matching in G w.h.p. The expected subset-query complexity is $\text{poly}(\log n) \cdot (1/\varepsilon)^{O(1/\varepsilon)}$.*

⁴This is parallel to Lemmas 3.4 - 3.6 of [\[YY109\]](#) for controlling the distortion in the uniform distribution.

Proof. We will call $\text{ESTIMATEMATCHINGSIZE}(G, \varepsilon, \eta = 2/\varepsilon^2)$ using the subset-query implementations of COMPUTEDEGREES and MATCHLOWDEGREE , where instead of returning the exact value

$$\hat{\mu} := \frac{1 - \varepsilon}{1 + \varepsilon} \cdot \frac{1}{2} \sum_{v \in V} x_v,$$

we will sample a random set S of $O(\frac{\log n}{\varepsilon^2})$ indices and compute

$$\hat{\mu}_S := \frac{1 - \varepsilon}{1 + \varepsilon} \cdot \frac{n}{2|S|} \sum_{v \in S} x_v,$$

then return $\hat{\mu}'_S := \hat{\mu}_S - \varepsilon n$. First we observe that we can choose the constant such that

$$\Pr[|\hat{\mu} - \hat{\mu}_S| > \varepsilon n] \leq n^{-c}.$$

This follows from the fact that $x_v \in [0, 1]$ for all v and a Hoeffding bound. Combined with the validity and approximation guarantees of [Corollary 3.7](#) and [Lemma 3.8](#), this yields an estimate $\hat{\mu}'_S$ such that

$$\frac{\mu(G)}{2 + O(\varepsilon n)} - 2\varepsilon n \leq \hat{\mu}'_S \leq \mu(G).$$

Since $\mu(G) < n$, we conclude the multiplicative-additive approximation guarantee:

$$\mu(G)/2 - O(\varepsilon n) \leq \hat{\mu}'_S \leq \mu(G),$$

where the $O(\varepsilon n)$ can be reduced to εn by scaling ε by the appropriate constant.

Now we bound the query complexity. For $v \in S$ such that v is removed from U during a call to PEELHIGHDEGREE , x_v can just be retrieved from the vector $\vec{x}$ without making a call to the local implementation of MATCHLOWDEGREE . For v that remains in U after all calls to PEELHIGHDEGREE , MATCHLOWDEGREE is called on a graph of degree at most $\eta = O(1/\varepsilon^2)$, so by [Lemma 4.6](#), x_v takes $(1/\varepsilon)^{O(1/\varepsilon)}$ subset queries to compute, in expectation over a random vertex.

Thus, combined with [Claim 4.8](#), the expected total number of subset queries is bounded by

$$\text{poly}(\log n) \cdot (1/\varepsilon)^{O(1/\varepsilon)},$$

which concludes the proof. $\square$

Remark 4.9. To conclude the stronger version of the theorem, we restart the algorithm if it has not returned within twice its expected running time, and make at most $O(\log n)$ attempts. This gives an algorithm that still succeeds w.h.p. and always has $\text{poly}(\log n) \cdot (1/\varepsilon)^{O(1/\varepsilon)}$ query complexity.

To go from a multiplicative-additive to a multiplicative approximation, we first get rid of singleton vertices in the low-degree case. After that, the maximum matching of the remaining vertices will be of size at least $\Omega(k/\Delta)$ where $\Delta = O(1/\varepsilon^2)$ is the maximum degree in the low-degree case and k is the number of non-singletons. Sampling $\log n \cdot (\varepsilon/\Delta)^{-2}$ vertices instead of $\log n/\varepsilon^2$ gives a multiplicative approximation. This technique is standard (see [\[Beh21\]](#)).

5 Streaming Distance-to-Monotonicity Estimation

In this section we prove [Theorem 1](#), giving a $\tilde{O}(n)$ -space, $\sqrt{n}^{1+o(1)}$ -pass streaming algorithm for distance-to-monotonicity estimation. We do this by giving a streaming implementation of a subset query in the violation graph G_f , which will allow us to use the subset-query algorithm to estimate the size of the maximum matching in the violation graph. From [Fact 2.3](#), it will follow that this gives an estimator for distance to monotonicity.

Lemma 5.1 (Streaming implementation of subset query). *A subset query in the conflict graph G_f can be implemented in $\sqrt{n}^{1+o(1)}$ passes over the edge and label stream of G, f and $\tilde{O}(n)$ space. The query succeeds with high probability.*

We cite the following result, originally presented by [\[JLS19\]](#) as a PRAM algorithm. See Proposition 4 of [\[AJJ⁺22\]](#) and the discussion beneath it for an explanation of how it is implemented in the streaming model.

Proposition 5.2 (Streaming shortcut set computation [\[JLS19, AJJ⁺22\]](#)). *There is a $\sqrt{n}^{1+o(1)}$ -pass, $\tilde{O}(n)$ -space algorithm that given directed $G = (V, E)$, with high probability computes a shortcut set⁵ $H \subset V \times V$ of $\tilde{O}(n)$ additional edges such that*

1. $\text{diam}(G \cup H) \leq \sqrt{n}^{1+o(1)}$, and
2. For all $(u, v) \in V \times V$, v is reachable from u in $G \cup H$ iff v is reachable from u in G .

Proof of Lemma 5.1. Let S be the queried subset. We will first use one pass and $O(n)$ space to store all the labels of f . Partition S into $S_0 := S \cap f^{-1}(0)$ and $S_1 := S \cap f^{-1}(1)$. We will then use $\sqrt{n}^{1+o(1)}$ passes and $\tilde{O}(n)$ space to build a $\sqrt{n}^{1+o(1)}$ -diameter shortcut set as specified in [Proposition 5.2](#). We will run a (directed) parallel BFS in $G \cup H$ from S_1 as follows:

1. For one pass, whenever a (directed) edge (u, v) appears in the stream where $u \in S_1$, place a mark on v .
2. For $\text{diam}(G \cup H)$ passes, whenever a (directed) edge (u, v) appears in the stream where u is marked, mark v .
3. Let T_1 be the set of marked vertices labeled 0.

We will then run the analogous BFS from S_0 :

1. For one pass, whenever a (directed) edge (u, v) appears in the stream where $v \in S_0$, place a mark on u .
2. For $\text{diam}(G \cup H)$ passes, whenever an edge (u, v) appears in the stream where v is marked, mark u .
3. Let T_0 be the set of marked vertices labeled 1.

⁵Some authors make a distinction between a shortcut set and a hopset, where the latter consists of additional *weighted* edges which approximately preserve shortest paths. For clarity, we use the term “shortcut set” for the unweighted concept instead of “hopset” as in [\[JLS19, AJJ⁺22\]](#).

First, observe that the BFS marks exactly the set of descendants of S_1 and ancestors of S_0 in $G \cup H$. By the fact that H preserves the reachability relation in G , these are also exactly the descendants and ancestors in G . A vertex u violates monotonicity with $v \in S_0$ iff $f(u) = 1$ and u is an ancestor of v , so u violates monotonicity with some $v \in S_0$ iff $u \in T_0$. Similarly, u violates monotonicity with $v \in S_1$ iff $f(u) = 0$ and u is a descendant of v , so u violates monotonicity with some $v \in S_0$ iff $u \in T_0$.

Thus, $T_0 \cup T_1$ is the correct answer to the subset query $S = S_0 \cup S_1$; i.e. it contains exactly the set of vertices adjacent to S in the conflict graph.

Since $\text{diam}(G \cup H) \leq \sqrt{n}^{1+o(1)}$ and we only need to store the marks, the number of passes required for the BFS is $\leq \sqrt{n}^{1+o(1)}$ and the space is $O(n)$. This gives a total of $\sqrt{n}^{1+o(1)}$ passes and $\tilde{O}(n)$ space for the entire subset query. $\square$

We will now complete the proof of [Theorem 1](#) by combining the pass complexity of this subset query implementation with the bound on the subset-query complexity. Rather than using the subset-query implementation exactly as presented in [Section 4](#), we simplify the low-degree stage and improve the ε dependence by taking advantage of the fact that once the degree falls low enough, we can build an explicit representation of G_f in one pass. So instead of using the local algorithm which makes $(1/\varepsilon)^{O(1/\varepsilon)}$ subset queries, we will build G_f and use the global algorithm [Algorithm 3](#) as written.

Proof of [Theorem 1](#). We will run ESTIMATEMATCHINGSIZE in the violation graph G_f using the subset-query implementation of COMPUTEDEGREES, and a single-pass global implementation of MATCHLOWDEGREE. Since only these functions require access to the edges of G_f , no passes are made outside of them.

We will first bound the space requirements of ESTIMATEMATCHINGSIZE and PEELHIGHDEGREE outside of the calls to COMPUTEDEGREES and MATCHLOWDEGREE. We maintain $\Delta_{\max}$, which requires $O(\log n)$ space, the set of active vertices U , which requires $O(n)$ space, and the saturation vector $\vec{x}$, which requires $O(n \log(1/\varepsilon))$ space to store weights of granularity $O(\varepsilon)$.

We will now bound the space complexity of MATCHLOWDEGREE. We will build the entire conflict graph $G_f[U]$ in one pass by storing all the edges and adding an edge between every reachable pair that violates monotonicity. Then we will run the rest of the subroutine on the stored graph, making no more passes. The graph at this point has degree at most $O(1/\varepsilon^2)$; thus the space complexity of the graph is $O(n \log n / \varepsilon^2)$. Each matching takes $O(n \log n)$ space, but the matchings are not stored between iterations — only U and $\vec{x}$ persist, which take $O(n \log(1/\varepsilon))$ space. This subroutine then takes $O(n \log n / \varepsilon^2)$ space in total.

Now we will analyze the space and pass complexity of COMPUTEDEGREES. By [Lemma 5.1](#), COMPUTEDEGREES takes $O(n \log(n/\varepsilon))$ space, so the total space complexity is still $\tilde{O}(n/\varepsilon^2)$.

By [Claim 4.8](#), the number of subset queries is bounded by $\text{poly}(\log n, 1/\varepsilon)$. Combined with [Lemma 5.1](#), this gives a total pass complexity of $\sqrt{n}^{1+o(1)} \cdot \text{poly}(1/\varepsilon)$. By [Lemmas 3.6](#) and [3.8](#), the output is a $(2 + O(\varepsilon))$ -approximation to the maximum matching size in G_f ; the standard technique of rescaling ε by the appropriate constant reduces this to a $(2 + \varepsilon)$ -approximation. By [Fact 2.3](#), it is thus a $(2 + \varepsilon)$ -approximation to the distance to monotonicity, from which the theorem follows. $\square$

6 Implementation With Vertex Queries

In this section we show how to adapt the ideas of [Sections 3 and 4](#) to the vertex-query model. The main difficulty in this model, compared to the subset-query model, is that accurate multiplicative degree estimates are not possible for low-degree vertices: for instance, identifying the set of degree-zero vertices in a graph would require $\Theta(n)$ vertex queries. This rules out directly implementing [Assumption 3.1](#). Instead, we use the following alternate assumption:

Assumption 6.1 ($\text{COMPUTEDEGREES}_\theta$). *There is a function $\text{COMPUTEDEGREES}_\theta(G, U, \varepsilon)$ that takes an arbitrary graph $G = (V, E)$, subset $U \subseteq V$, and approximation parameter $\varepsilon > 0$, and returns a vector $\vec{d}$ such that:*

- d_v is nonzero for all $v \in V$ with $|N(v) \cap U| \geq \theta$, and zero for all $v \in V$ with $|N(v) \cap U| \leq (1 - \varepsilon)\theta$.
- If d_v is nonzero, then $d_v = (1 \pm \varepsilon)|N(v) \cap U|$.

We will show in [Lemma 6.5](#) that it is possible to implement $\text{COMPUTEDEGREES}_\theta$ for $\theta \approx \sqrt{n}$ using about $\sqrt{n}$ vertex queries. This will allow us to run PEELHIGHDEGREE with degree thresholds above $\sqrt{n}$; however it will not allow us to drop the degree all the way to $2/\varepsilon^2$ as we do in the subset query implementation. Since we must transition to the low-degree phase at a threshold $\eta \approx \sqrt{n}$, it will also no longer be feasible to use the result of [Lemma 4.6](#), as each x_v would take $\eta^{O(1/\varepsilon)} = n^{O(1/\varepsilon)}$ queries to compute.

To eliminate the $O(1/\varepsilon)$ in the exponent, we switch from computing an adaptive sequence of $1/\varepsilon$ maximal integral matchings to computing a single maximal fractional matching, which increases the integrality gap from $1/(1 - \varepsilon)$ to $3/2$. Thus, at the end of the algorithm, we will scale the estimate by $2/3$ instead of $1 - \varepsilon$, achieving an approximation ratio of 3.

In summary, the modified global algorithm $\text{ESTIMATEMATCHINGSIZEVQ}$ differs from [Algorithm 1](#) in four ways:

1. The transition from high to low degrees is defined as $\eta = \sqrt{n}$ instead of $2/\varepsilon^2$.
2. The PEELHIGHDEGREE subroutine uses $\text{COMPUTEDEGREE}_\theta$ with parameter $\theta = \frac{\varepsilon^2 \eta}{2 \ln n}$ in place of COMPUTEDEGREE .
3. The MATCHLOWDEGREE subroutine is replaced with MATCHLOWDEGREEVQ defined in [Algorithm 8](#).
4. The final estimate returned by the algorithm is $\hat{\mu} = \frac{1}{1+3\varepsilon} \cdot \frac{1}{3} \sum_v x_v$ instead of $\frac{1-\varepsilon}{1+\varepsilon} \cdot \frac{1}{2} \sum_v x_v$.

Algorithm 8: MATCHLOWDEGREEVQ($G, U, \vec{x}, \varepsilon$)

Input: Graph $G = (V, E)$, active set U , saturation vector $\vec{x}$, approximation parameter $\varepsilon > 0$

Output: Updated saturation vector

```
1  $V' \leftarrow V \times [1/\varepsilon]$ 
2  $E' \leftarrow \{ \{(v_1, i_1), (v_2, i_2)\} : \{v_1, v_2\} \in G \}$ 
3  $G' \leftarrow (V', E')$ 
4  $U' \leftarrow \{(v, i) \in V' \times \{0, \dots, 1/\varepsilon\} : v \in U \wedge \varepsilon i \leq x_v\}$ 
5  $M \leftarrow \text{MAXIMALMATCHING}(G', U')$ 
6 for  $\{(u, i_1), (v, i_2)\} \in M$  do
7    $x_u \leftarrow x_u + \varepsilon$ 
8    $x_v \leftarrow x_v + \varepsilon$ 
   /*  $y_{uv} \leftarrow y_{uv} + \varepsilon$  */
9 return  $\vec{x}$ 
```

6.1 Correctness and Approximation Ratio of the Modified Global Algorithm

Now we argue that the correctness conditions of [Section 3](#) apply to the modified algorithm, with slight changes.

Lemma 6.2. *After the last call to PEELHIGHDEGREE in ESTIMATEMATCHINGSIZEVQ, the vectors $\vec{x}, \vec{y}$ satisfy the three conditions of [Lemma 3.3](#), except that the first condition is replaced by $(1 - \varepsilon) \left[\sum_{u \in N(v)} y_{uv} \right] - \varepsilon \leq x_v \leq (1 + \varepsilon) \sum_{u \in N(v)} y_{uv}$.*

Proof. The behavior of PEELHIGHDEGREE is almost the same under [Assumption 6.1](#) instead of [Assumption 3.1](#). The identity of vertices in H is unchanged, as is the approximation guarantee on $(d_U)_v$ for $v \in H$. The only difference is that $(d_H)_v$ may be 0 for vertices $v \in U \setminus H$ with fewer than θ neighbors in U . For those vertices, the value of x_v may be too small, as compared to the original PEELHIGHDEGREE. It immediately follows that conditions 2 and 3 as well as the upper bound of condition 1 are satisfied.

Now we bound the amount of error in x_v . The size of the error accumulates by at most $\frac{\varepsilon}{2} \cdot \frac{\theta}{\Delta_{\max}} \leq \frac{\varepsilon \theta}{2\eta}$ in each of at most $4/\varepsilon$ iterations of PEELHIGHDEGREE, and thus the cumulative error across at most $\frac{\ln n}{\varepsilon}$ calls to PEELHIGHDEGREE is bounded by $\frac{2\theta \ln n}{\varepsilon \eta}$, which is at most ε by choice of θ . $\square$

Lemma 6.3 (VQ analogue of [Corollary 3.7](#) (validity)). *Assume $\varepsilon \leq \frac{1}{3}$, and let $\vec{x}$ be the vector computed by ESTIMATEMATCHINGSIZEVQ ([Algorithm 1](#)). Then there exists a matching in G of value*

$$\hat{\mu} = \frac{1}{1+3\varepsilon} \cdot \frac{1}{3} \sum_v x_v.$$

Proof. First we will show that there exists a fractional matching in G with vertex saturation vector $\frac{1}{1+3\varepsilon} \vec{x}$. It follows from [Lemma 6.2](#) and the behavior of [Algorithm 8](#) that the final values $\vec{x}$ and $\vec{y}$ satisfy $\frac{1}{1+\varepsilon} x_v \leq \sum_{u \in N(v)} y_{uv} \leq 1 + \varepsilon$ for every v ; thus $\frac{1}{(1+\varepsilon)} \vec{y}$ is a fractional matching with vertex saturations at least $\frac{1}{(1+\varepsilon)^2} \vec{x} \geq \frac{1}{1+3\varepsilon} \vec{x}$.

This fractional matching has value at least $\frac{1}{1+3\varepsilon} \cdot \frac{1}{2} \sum_v x_v$. By [Fact 2.4](#), there is then an integral matching of value at least

$$\frac{2}{3} \cdot \frac{1}{1+3\varepsilon} \cdot \frac{1}{2} \sum_v x_v = \frac{1}{1+3\varepsilon} \cdot \frac{1}{3} \sum_v x_v.$$

□

Lemma 6.4 (VQ analogue of [Lemma 3.8](#) (approximation ratio)). *The fractional matching value $\hat{\mu}$ returned by ESTIMATEMATCHINGSIZEVQ satisfies*

$$\hat{\mu} \geq \frac{\mu(G)}{3 + O(\varepsilon)}.$$

Proof. We observe that every edge $(u, v) \in G$ satisfies $x_u + x_v \geq 1 - O(\varepsilon)$. Let L be the set of vertices in U when MATCHLOWDEGREEVQ is called; the case where u or v is in $V \setminus L$ is identical to the analogous case in the proof of [Lemma 3.8](#). When u and v are both in L , this condition follows from the maximality of M in MATCHLOWDEGREEVQ. The lemma statement then follows from [Fact 3.9](#). □

6.2 Implementing the Modified Algorithm Using Vertex Queries

We start by showing that [Assumption 6.1](#) can be implemented in the vertex query model.

Algorithm 9: Implementing COMPUTEDEGREES $_{\theta}$ via vertex queries

Input: Graph $G = (V, E)$ with $|V| = n$, subset $U \subseteq V$, threshold $\theta > 0$, approximation parameter $\varepsilon > 0$, failure probability $\delta > 0$

Output: Degree estimates $\vec{d}$

```

1  $r \leftarrow C\varepsilon^{-2} \frac{|U|}{\theta} \log(n\delta^{-1})$  for a sufficiently large constant  $C$ 
2  $\vec{X} \leftarrow 0$ 
3 repeat  $r$  times
4   Choose  $u \in U$  uniformly at random
5   for  $v \in N(u)$  do
6      $X_v \leftarrow X_v + 1$ 
7 for  $v \in V$  do
8    $d_v \leftarrow \frac{|U|}{r} X_v$ 
9   if  $d_v \leq (1 - \varepsilon/2)\theta$  then
10     $d_v \leftarrow 0$ 
11 return  $\vec{d}$ 
```

Lemma 6.5 (Degree estimation via vertex queries). *The functionality of [Assumption 6.1](#) can be implemented using $O\left(\varepsilon^{-2} \frac{|U|}{\theta} \log(n\delta^{-1})\right)$ vertex queries, with a failure probability of δ .*

Proof. Refer to [Algorithm 9](#). Fix a vertex $v \in V$ and consider X_v , the number of times v appears as a neighbor of one of the queried vertices. Then $\mathbf{E}X_v = \frac{r}{|U|} \cdot |N(v) \cap U|$. If $|N(v) \cap U| \geq \theta/4$ then by a Chernoff bound,

$$\Pr[|X_v - \mathbf{E}X_v| \geq (\varepsilon/3)|\mathbf{E}X_v|] \leq 2 \exp\left(-\frac{(\varepsilon/3)^2}{3} \mathbf{E}X_v\right)$$

$$\leq n^{-1}\delta,$$

provided C is sufficiently large. On the other hand, if $|N(v) \cap U| \leq \theta/4$ then

$$\begin{aligned} \Pr \left[X_v \geq \frac{3}{4} \cdot \frac{r}{|U|} \cdot \theta \right] &\leq \exp \left(-\frac{r}{|U|} \theta/4 \right) \\ &\leq n^{-1}\delta, \end{aligned}$$

again provided C is sufficiently large. (This uses the fact that the one-sided Chernoff upper bound holds with $\mathbf{E}X_v$ replaced by the upper bound $\frac{r}{|U|}\theta/4$, which follows from a coupling argument.) Thus by taking a union bound over all $v \in V$, with probability at least $1 - \delta$ the returned $\vec{d}$ correctly distinguishes vertices with at least θ neighbors in U from those with at most $(1 - \varepsilon)\theta$, and correctly estimates degrees whenever it returns a nonzero value. $\square$

Now we use the following result of [Beh21] to implement query access to the output of MAXIMALMATCHING.

Theorem 6 ([Beh21, Theorem 3.5]). *Let $G = (V, E)$ be a graph with average degree $\bar{d}$. Let Π be the set of all permutations over E . For a permutation $\pi \in \Pi$. For a vertex $v \in V$, there is a randomized oracle $\text{VO}(v, \pi)$ that determines whether v is matched in $\text{GMM}(G, \pi, e)$ (Algorithm 5) and, if so, identifies the matching edge. Let $T(v, \pi)$ denote the total number of recursive calls to the edge oracle generated during the execution of $\text{VO}(v, \pi)$. Then for a vertex v chosen uniformly at random from V and a permutation π chosen uniformly at random from Π , independently from v ,*

$$\mathbb{E}_{v \sim V, \pi \sim \Pi}[T(v, \pi)] = O(\bar{d} \cdot \log n).$$

Lemma 6.6. *There exists a randomized vertex-query algorithm that computes $\varepsilon n \pm \sum_v x'_v$, where x'_v is the output of MATCHLOWDEGREEVQ (Algorithm 8) on inputs $G, U, \vec{x}, \varepsilon$, using vertex queries to G . The algorithm uses $O(\bar{d}_U \cdot \varepsilon^{-3}(\log n)(\log \delta^{-1}))$ vertex queries in expectation and succeeds with probability $1 - \delta$, where $\bar{d}_U$ is the average degree in $G[U]$.*

Proof. Let $G' = (V', E')$ and $U' \subset V'$ be as in MATCHLOWDEGREEVQ .

It suffices to estimate to within an additive error of ε the proportion p of vertices in V' which are matched by a maximal matching of $G'[U']$, since given such an estimate $\hat{p}$ we can output $\hat{p}n + \sum_{v \in V'} x_v = \varepsilon n \pm \sum_{v \in V'} x'_v$. We can accomplish this by sampling $\Theta(\varepsilon^{-2} \log \delta^{-1})$ random vertices and determining if they are matched.

Draw a permutation π uniformly at random over $E(G'[U'])$ and let M be the matching computed by the algorithm of Theorem 6 on input $G'[U']$.

Pick a vertex $v \in V'$ uniformly at random. If $v \notin U'$ then it is not matched, so assume it is in U' . We run the oracle $\text{VO}(v, \pi)$ on $G'[U']$, implementing the oracle's adjacency-list access to $G'[U']$ using vertex queries to G . The expected total number of vertex queries is then $O(\bar{d}_{G'[U']} \cdot \varepsilon^{-2}(\log n)(\log \delta^{-1})) = O(\bar{d}_U \cdot \varepsilon^{-3}(\log n)(\log \delta^{-1}))$. $\square$

6.3 Proof of Theorem 2

Finally, we will combine the results of the previous subsections and bound the total number of vertex queries, obtaining Theorem 2. As before, we will bound the expected number of queries; the same repetition technique discussed in Remark 4.9 applies. (The multiplicative error technique does not apply due to the fact that we cannot efficiently detect singleton vertices using vertex queries.)

Proof of Theorem 2. From Lemma 6.6 we obtain an estimate $\hat{x} = \varepsilon n \pm \sum_{v \in S} x_v$, from which we compute and return

$$\hat{\mu}_S := \frac{1}{1 + 3\varepsilon} \cdot \frac{1}{3} \hat{x} - \varepsilon n.$$

Then this is a $(3, \varepsilon n)$ -approximation according to Lemmas 6.3 and 6.4.

Now we bound the number of vertex queries in the high-degree phase. Vertex queries in this phase are only made during calls to COMPUTEDEGREES. By the argument in the proof of Claim 4.8, COMPUTEDEGREES is called $O((\log n)/\varepsilon)$ times. By Lemma 6.5, the query complexity of COMPUTEDEGREES $_\theta$ with failure probability n^{-c} is

$$O\left(\frac{n \log n}{\varepsilon^2 \theta}\right).$$

We are calling it with

$$\theta = \frac{\varepsilon^2 \sqrt{n}}{2 \ln n},$$

therefore the total query complexity of all calls to COMPUTEDEGREES is

$$O\left(\frac{n \log^2 n}{\varepsilon^4 \sqrt{n}} \cdot \frac{\log n}{\varepsilon}\right) = O\left(\frac{\sqrt{n} \log^3 n}{\varepsilon^5}\right) = \sqrt{n} \cdot \text{poly}(\log n, \varepsilon^{-1}).$$

Finally, the number of vertex queries in the low-degree phase is bounded by $\bar{d}_L \cdot \text{poly}(\log n, \varepsilon^{-1}) \leq \sqrt{n} \cdot \text{poly}(\log n, \varepsilon^{-1})$ according to Lemma 6.6, so this is the query complexity for the algorithm as a whole. $\square$

7 Better Approximation Ratios with Subset Queries

In this section, we show that if we restrict our attention to subset queries, then we can outperform Algorithm 1 in terms of approximation ratio. We remark that Algorithm 1 applies under weaker guarantees, namely those of the COMPUTEDEGREES $_\theta$ subroutine in Section 6, and can therefore be implemented efficiently using either subset or vertex queries. This flexibility comes at the cost of a larger approximation ratio. In contrast, the algorithm in this section requires stronger guarantees (as in Assumption 3.1) that are achievable only through subset queries.

We prove

Theorem 7. *Given an n -vertex graph G , for any $\varepsilon > 0$, there is a randomized algorithm that $(1.5 + \varepsilon)$ -approximates the size of maximum matching in G using $\text{poly}(\log n, 1/\varepsilon)$ subset queries to G w.h.p. If the graph is bipartite, then the approximation ratio improves to $1 + \varepsilon$.*

This is achieved through a $(1 + \varepsilon)$ -approximation of the minimum *fractional* vertex cover, whose value is equal to the maximum fractional matching. In turn, this yields a $(1.5 + \varepsilon)$ -approximation for general (i.e., possibly non-bipartite) graphs, due to the 1.5 integrality gap, and a $(1 + \varepsilon)$ -approximation for bipartite graphs.

Recall that, for a Boolean function f on a DAG G , the conflict graph G_f satisfies $d_{\text{mon}}(f) = \mu(G_f)$ (fact 2.3). Therefore, since G_f is bipartite, and a subset query to G_f can be implemented in $\sqrt{n}^{1+o(1)}$ (Lemma 5.1) passes and space $\tilde{O}(n/\varepsilon^2)$, we have:

Theorem 8. *For any $\varepsilon > 0$, there is a randomized streaming algorithm that takes $\sqrt{n}^{1+o(1)} \cdot \text{poly}(1/\varepsilon)$ passes over the stream, uses $\tilde{O}(n/\varepsilon^2)$ space, and w.h.p. $(1 + \varepsilon)$ -approximates $d_{\text{mon}}(f)$.*

The algorithm relies on the COMPUTEDEGREES subroutine, as defined in [Assumption 3.1](#), which produces a $(1 + \varepsilon)$ -approximation of all the vertex degrees using $O\left(\frac{\log^2 n}{\varepsilon^3}\right)$ subset queries ([Lemma 4.1](#)). We adapt Lemma 3.5 of Liu [[Liu24](#)], which implements a multiplicative-weight-update (MWU) oracle using $O(\varepsilon^{-2} \log n)$ calls to COMPUTEDEGREES. Then, the value of the minimum fractional vertex cover can be $(1 + \varepsilon)$ -approximated using $T = O(\varepsilon^{-2} \log n)$ calls to the MWU oracle.

Definitions

We begin with some definitions. Given a graph $G = (V, E)$ a *fractional vertex cover* is a set of values $y \in \mathbb{R}_{\geq 0}^V$, such that

$$y_u + y_v \geq 1 \quad \text{for every } (u, v) \in E.$$

We use $\tau_f(G)$ to denote the value of the minimum fractional vertex cover, which is the minimum possible value of $\|y\|_1$ among all fractional vertex covers y .

For the MWU framework, we consider parameters $c, \lambda, \varepsilon > 0$, and define a potential function $\Phi : \mathbb{R}_{\geq 0}^V \rightarrow \mathbb{R}$,

$$\Phi(y) = \sum_{(u,v) \in E} \exp(-\lambda(y_u + y_v)).$$

For a fixed y , we similarly define s_v to be the sum of the terms in the potential corresponding to edges incident to v :

$$s_v = \sum_{u \in N(v)} \exp(-\lambda(y_u + y_v)).$$

Definition 7.1 (MWU oracle). *Given a graph $G = (V, E)$, $y \in \mathbb{R}_{\geq 0}^V$, and $c, \lambda, \varepsilon > 0$, an MWU oracle either fails or outputs an update vector $\Delta \in \mathbb{R}_{\geq 0}^V$ such that:*

1. $\sum_v \Delta_v = 1$,
2. $\sum_v \Delta_v s_v \geq (1 - \varepsilon)c \cdot \Phi(y)$, and
3. $\Delta_v \leq \frac{\varepsilon}{2\lambda}$ for all $v \in V$.

When $\frac{1}{2\tau_f(G)} \leq c \leq \frac{1}{\tau_f(G)}$, the oracle is guaranteed to succeed.

The MWU Framework

First, we give an overview of how an MWU oracle can be used to obtain a $(1 + \varepsilon)$ -approximation of the minimum fractional vertex cover. The reader may refer to [[Liu24](#), Section 3] for a full proof. We remark that their proof is stated for $\mu(G)$, the size of the maximum matching, in bipartite graphs. The only modification in our setting appear is in [Claim 7.3](#), where we show how the MWU oracle is obtained from COMPUTEDEGREES.

Claim 7.2 (See [[Liu24](#), Section 3]). *Given a graph G , for $\lambda = \varepsilon/10c$, a $(1 + \varepsilon)$ -approximate minimum fractional vertex cover can be computed using $\text{poly}(\log n, 1/\varepsilon)$ calls to the MWU oracle.*

Proof sketch. First, observe that the approximation problem can be reduced to a decision problem as follows: For $O(\frac{\log n}{\varepsilon})$ choices of $c = (1 + \varepsilon)^{-i}$, we test whether there exists a $y \in \mathbb{R}_{\geq 0}^V$ such that

$$\sum_v y_v = 1 \quad \text{and} \quad y_u + y_v \geq c, \quad \forall (u, v) \in E. \quad (3)$$

This is feasible if and only if $c \leq 1/\tau_f(G)$. Therefore, the largest feasible guess yields a $(1 + \varepsilon)$ -approximation of $\tau_f(G)$.

On a high level, the decision problem is solved as follows. Let $\lambda = \varepsilon/10c$ and $T = 20\varepsilon^{-2} \log n$. Starting with $y^{(0)} = 0$, for $t = 0, 1, \dots, T-1$, let $y^{(t+1)} = y^{(t)} + \Delta^{(t)}$, where $\Delta^{(t)}$ is the output of the MWU oracle for $y^{(t)}$. It can be shown that when the oracle succeeds in every step, $\bar{y} = y^{(T)}/T$ satisfies (3) up to a $1 + O(\varepsilon)$ factor. Hence, rescaling $\bar{y}$ gives a fractional vertex cover of size $(1 + O(\varepsilon))\frac{1}{c}$. As a result, considering the largest c for which the oracle succeeds in every step, yields the $(1 + O(\varepsilon))$ -approximation. We remark that the oracle may behave arbitrarily when $c < \frac{1}{2\tau_f(G)}$. $\square$

Therefore, it suffices to show that an MWU oracle can be obtained using COMPUTEDEGREES. We make the necessary adaptations for general graphs and $\tau_f(v)$.

Claim 7.3 (Analogous to [Liu24, Lemma 3.5]). *The MWU oracle, as defined in Definition 7.1, can be implemented using $O(\varepsilon^{-2} \log n)$ calls to COMPUTEDEGREES.*

Proof. First, we obtain estimates $\tilde{s}_v$ for s_v for all vertices v :

$$(1 - \varepsilon/2)s_v \leq \tilde{s}_v \leq s_v, \quad (4)$$

where recall that

$$s_v = \sum_{u \in N(v)} e^{-\lambda(y_u + y_v)} = e^{-\lambda y_v} \sum_{u \in N(v)} e^{-\lambda y_u}.$$

To do so, we partition the vertices V into sets $V_0, \dots, V_{5T}$ where

$$V_i = \{v \in V \mid i\varepsilon/10 \leq \lambda y_v < (i+1)\varepsilon/10\}.$$

Note that $5T$ suffices as the maximum index, since by the construction of y , we have $y_v \leq T \cdot \frac{\varepsilon}{2\lambda}$ (due to Item 3 of Definition 7.1). Then, we estimate $\sum_{u \in N(v) \cap V_i} e^{-\lambda y_u}$ by $|N(v) \cap V_i| e^{-i\varepsilon/10}$, where $|N(v) \cap V_i|$ can be estimated simultaneously for all v , up to a $(1 + \varepsilon)$ factor using COMPUTEDEGREES. Formally, letting $\tilde{d}_{v,i}$ be the $(1 + \varepsilon)$ -approximation for $|N(v) \cap V_i|$, we have:

$$\begin{aligned} \sum_i \tilde{d}_{v,i} e^{-i\varepsilon/10} &= (1 \pm O(\varepsilon)) \sum_i |N(v) \cap V_i| e^{-i\varepsilon/10} \\ &= (1 \pm O(\varepsilon)) \sum_i \sum_{u \in N(v) \cap V_i} e^{-\lambda y_u} \\ &= (1 \pm O(\varepsilon)) \sum_{u \in N(v)} e^{-\lambda y_u}. \end{aligned}$$

Multiplying this value by $\frac{e^{-\lambda y_v}}{1 + O(\varepsilon)}$ and rescaling ε appropriately, yields the desired estimate $\tilde{s}_v$.

Next, we use these estimates $\tilde{s}_v$ to construct Δ . Sort the vertices $v_1, v_2, \dots, v_n$ in decreasing order of $\tilde{s}_v$. Let U be the largest prefix that satisfies:

$$\frac{1}{|U|} \sum_{v \in U} \tilde{s}_v \geq (1 - \varepsilon)c\Phi(y). \quad (5)$$

Then, let

$$\Delta_v = \begin{cases} 1/|U|, & \text{if } v \in U, \text{ and} \\ 0, & \text{otherwise.} \end{cases}$$

Observe that [Item 1](#) of [Definition 7.1](#) is satisfied trivially by the choice of Δ , and [Item 2](#) is satisfied due to (4) and (5). When $|U| < \frac{2\lambda}{\varepsilon}$, the algorithm declares failure. When $c \in \left[\frac{1}{2\tau_f(G)}, \frac{1}{\tau_f(G)}\right]$, we show that $|U| \geq \frac{2\lambda}{\varepsilon}$ and hence [Item 3](#) holds.

We show that $|U| \geq \lfloor \tau_f(G) \rfloor$. In turn, it follows that

$$|U| \geq \frac{\tau_f(G)}{2} \geq \frac{1}{4c} \geq \frac{2\lambda}{\varepsilon}.$$

We argue that the prefix of size $k = \lfloor \tau_f(G) \rfloor$ satisfies (5). Let $x \in \mathbb{R}_{\geq 0}^V$ be an optimal fractional minimum vertex cover. Consider $\sum_v x_v \tilde{s}_v$ as a weighted sum of $\{\tilde{s}_v\}_v$. Since $x_v \leq 1$ for all v , the sum can only grow, by concentrating the weight on the k vertices with the largest $\tilde{s}_v$. That is, giving a weight of $\frac{\tau_f(G)}{k}$ to the first k vertices:

$$\sum_v x_v \tilde{s}_v \leq \frac{\tau_f(G)}{k} \sum_{i=1}^k \tilde{s}_{v_i}. \quad (6)$$

On the other hand, using the fact that x is a vertex cover, we get:

$$\begin{aligned} \sum_v x_v \tilde{s}_v &\geq (1 - \varepsilon/2) \sum_v x_v s_v \\ &= (1 - \varepsilon/2) \sum_{(u,v) \in E} e^{-\lambda(y_u + y_v)} (x_u + x_v) \\ &\geq (1 - \varepsilon/2) \Phi(y). \end{aligned} \quad (7)$$

Putting (6) and (7) together, we get:

$$\frac{1}{k} \sum_{i=1}^k \tilde{s}_{v_i} \geq \frac{1 - \varepsilon/2}{\tau_f(G)} \Phi(y) \geq (1 - \varepsilon/2)c\Phi(y).$$

Therefore, $|U| \geq \lfloor \tau_f(G) \rfloor$, which concludes the proof of [Item 3](#). We finally remark that, since there are $5T = O(\varepsilon^{-2} \log n)$ vertex sets, the oracle requires as many calls to `COMPUTEDEGREES` for one guess c . $\square$

To conclude, we remark that [Claims 7.2](#) and [7.3](#) together imply that a $(1 + \varepsilon)$ -approximation of the minimum fractional vertex cover can be computed using $\text{poly}(\log n, 1/\varepsilon)$ subset queries. Hence, [Theorem 7](#) follows which, in turn, implies [Theorem 8](#).

8 The Lower Bound

In this section, we establish a lower bound on the pass complexity of approximating the distance to monotonicity in the streaming model, which is stated in the following theorem.

Theorem 9. *For any fixed $c \geq 1$ there is $\varepsilon > 0$ such that a $(c, \varepsilon n)$ -approximation of $d_{\text{mon}}(f)$ with $\tilde{O}(n)$ space requires $\Omega(\text{st-reach}_n)$ passes, where st-reach_n denotes the pass-complexity of the best $\tilde{O}(n)$ space streaming algorithm for the st -reachability problem.*

Despite extensive studies, all we know about st-reach_n is that

$$\Omega\left(\frac{\log n}{(\log \log n)^2}\right) \stackrel{[\text{CKP}^+21]}{\leq} \text{st-reach}_n \stackrel{[\text{JLS19}]}{\leq} n^{1/2+o(1)}.$$

Note that Theorem 9 can also be seen as a conditional lower bound. It implies that Theorem 1 has optimal pass-complexity modulo improving the state-of-the-art st -reachability algorithm of [JLS19].

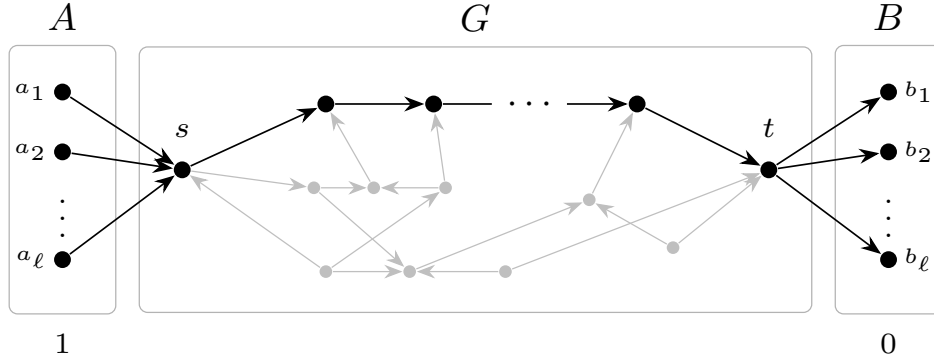


Figure 1: If t is reachable from s the instance is far from monotone.

Let $G = (V, E)$ be a directed graph with $|V| = n$, $|E| = m$, and vertices $s, t \in V$ be an instance of st -reachability. We construct a directed graph $G' = (V', E')$ and a boolean function $f: V' \rightarrow \{0, 1\}$ as follows. Let $l = 2cn$, for a fixed constant c . Introduce two fresh batches of vertices

$$A = \{a_1, \dots, a_l\}, \quad B = \{b_1, \dots, b_l\},$$

disjoint from V and from each other, and set $V' = A \cup V \cup B$ with $|V'| = 2l + n$. The edge set E' consists of (a, s) for every $a \in A$, (u, v) for every $(u, v) \in E$ and, (t, b) for every $b \in B$. Finally, define $f: V' \rightarrow \{0, 1\}$ by

$$f(v) = \begin{cases} 1 & v \in A, \\ 0 & v \in B, \\ 0 & v \in V. \end{cases}$$

First, we investigate the distance to monotonicity in the following two cases. The first case is when t is reachable from s , we call this the yes case, represented in Figure 1. The no case is when t is not reachable from s .

Lemma 8.1. *If s can reach t in G , then $d_{\text{mon}}(f) \geq 2cn$.*

Proof. Fix a directed path s to t in G . For every $(a, b) \in A \times B$, there is a directed path a to b in G' via the path from s to t , so a can reach b while $f(a) = 1 > 0 = f(b)$. All l^2 pairs in $A \times B$ are violations.

Any monotone function g must fix at least one endpoint per violation. Since these violations form the complete bipartite graph, any vertex cover has size at least l , so in G' $d_{\text{mon}}(f) \geq l = 2cn$. $\square$

Lemma 8.2. *If s cannot reach t in G , then $d_{\text{mon}}(f) \leq n$.*

Proof. Let g be a monotone function that agrees with f on all but at most n vertices. Let $R_s \subseteq V$ be the set of vertices reachable from s in G (including s itself). Define g as follows:

$$g(v) = \begin{cases} 1 & v \in A \cup R_s, \\ 0 & \text{otherwise.} \end{cases}$$

We verify that g is monotone on G' , i.e., for every edge $(x, y) \in E'$ we have $g(x) \leq g(y)$. For edges (a, s) where $a \in A$ we have $g(a) = 1$ and $g(s) = 1$ since $s \in R_s$. For edges $(u, v) \in E$ where $u \in R_s$, we have $v \in R_s$ as well, so $g(u) = g(v) = 1$. For edges $(u, v) \in E$ where $u \notin R_s$ we have $g(u) = 0 \leq g(v)$. For edges (t, b) where $b \in B$, since s cannot reach t , we have $t \notin R_s$, so $g(t) = 0 = g(b)$.

Therefore, the function g differs from f only on $R_s \subseteq V$ where $f = 0$ but $g = 1$, so $d_{\text{mon}}(f) \leq |R_s| \leq n$. $\square$

Proof of Theorem 9. Set $\varepsilon = \frac{c}{2(4c+1)}$, which satisfies $\varepsilon > 0$ for any fixed $c \geq 1$. Suppose $\mathcal{A}$ is a streaming algorithm that computes a $(c, \varepsilon N)$ -approximation of $d_{\text{mon}}(f)$ on posets of size N in p passes using space $\tilde{O}(N)$. The edges in G arrive in the same sequence as the stream for the s - t reachability problem. Assume f arrives in the beginning of the stream.

Let x be the value returned by running $\mathcal{A}$ on the stream for G' . If t is reachable from s , by Lemma 8.1, $d_{\text{mon}}(f) \geq 2cn$. Since $\mathcal{A}$ is a $(c, \varepsilon N)$ -approximation, its output must satisfy $x \geq d_{\text{mon}}(f) \geq 2cn$.

If t is not reachable from s , by Lemma 8.2, $d_{\text{mon}}(f) \leq n$. The algorithm $\mathcal{A}$ must output a value $x \leq c \cdot d_{\text{mon}}(f) + \varepsilon N$. Since $N = (4c + 1)n$, given our choice of ε we have $\varepsilon N = \frac{c}{2(4c+1)}(4c + 1)n = 0.5cn$. Thus, $x \leq cn + 0.5cn = 1.5cn$.

Because $1.5cn < 2cn$, the output of $\mathcal{A}$ allows us to perfectly distinguish whether s can reach t in G . Since $N = \Theta(n)$ and $|E'| = m + 2l = m + \Theta(n)$, generating the stream for G' requires only $\tilde{O}(n)$ space. Therefore, $\mathcal{A}$ effectively solves st -reachability in p passes and $\tilde{O}(n)$ space, implying that $p = \Omega(\text{st-reach}_n)$. $\square$

Acknowledgments

We thank an anonymous reviewer for pointing out the alternative approach detailed in Section 7.

References

- [ABR24] Amir Azarmehr, Soheil Behnezhad, and Mohammad Roghani. Fully dynamic matching: $(2 - \sqrt{2})$ -approximation in polylog update time. In David P. Woodruff, editor,

Proceedings of the 2024 ACM-SIAM Symposium on Discrete Algorithms, SODA 2024, Alexandria, VA, USA, January 7-10, 2024, pages 3040–3061. SIAM, 2024.

- [ABRR25] Amir Azarmehr, Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinfeld. Tight pair query lower bounds for matching and earth mover’s distance. In *66th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2025, Sydney, Australia, December 14-17, 2025*, pages 2666–2693. IEEE, 2025.
- [ACCL07] Nir Ailon, Bernard Chazelle, Seshadhri Comandur, and Ding Liu. Estimating the distance to a monotone function. *Random Struct. Algorithms*, 31(3):371–383, 2007.
- [ACCL08] Nir Ailon, Bernard Chazelle, Seshadhri Comandur, and Ding Liu. Property-preserving data reconstruction. *Algorithmica*, 51(2):160–182, 2008.
- [AJJ⁺22] Sepehr Assadi, Arun Jambulapati, Yujia Jin, Aaron Sidford, and Kevin Tian. Semi-streaming bipartite matching in fewer passes and optimal space. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 627–669. SIAM, 2022.
- [AKK25] Sepehr Assadi, Sanjeev Khanna, and Peter Kiss. Improved bounds for fully dynamic matching via ordered Ruzsa-Szemerédi graphs. In Yossi Azar and Debmalya Panigrahi, editors, *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2025, New Orleans, LA, USA, January 12-15, 2025*, pages 2971–2990. SIAM, 2025.
- [BB15] Aleksandrs Belovs and Eric Blais. Quantum algorithm for monotonicity testing on the hypercube. *Theory Comput.*, 11:403–412, 2015.
- [BB21] Aleksandrs Belovs and Eric Blais. A polynomial lower bound for testing monotonicity. *SIAM J. Comput.*, 50(3), 2021.
- [BCGM12] Jop Briët, Sourav Chakraborty, David García-Soriano, and Arie Matsliah. Monotonicity testing and shortest-path routing on the cube. *Comb.*, 32(1):35–53, 2012.
- [BCS18] Hadley Black, Deeparnab Chakrabarty, and C. Seshadhri. A $o(d) \cdot \text{polylog } n$ monotonicity tester for boolean functions over the hypergrid $[n]^d$. In Artur Czumaj, editor, *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2018, New Orleans, LA, USA, January 7-10, 2018*, pages 2133–2151. SIAM, 2018.
- [BCS20] Hadley Black, Deeparnab Chakrabarty, and C. Seshadhri. Domain reduction for monotonicity testing: A $o(d)$ tester for boolean functions in d -dimensions. In Shuchi Chawla, editor, *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA 2020, Salt Lake City, UT, USA, January 5-8, 2020*, pages 1975–1994. SIAM, 2020.
- [BCS23] Hadley Black, Deeparnab Chakrabarty, and C. Seshadhri. A $d^{1/2+o(1)}$ monotonicity tester for boolean functions on d -dimensional hypergrids. In *64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*, pages 1796–1821. IEEE, 2023.

- [Beh21] Soheil Behnezhad. Time-optimal sublinear algorithms for matching and vertex cover. In *62nd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2021, Denver, CO, USA, February 7-10, 2022*, pages 873–884. IEEE, 2021.
- [Beh23] Soheil Behnezhad. Dynamic algorithms for maximum matching size. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 129–162. SIAM, 2023.
- [BG24] Soheil Behnezhad and Alma Ghafari. Fully dynamic matching and ordered Ruzsa-Szemerédi graphs. In *65th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2024, Chicago, IL, USA, October 27-30, 2024*, pages 314–327. IEEE, 2024.
- [BGJ⁺10] Arnab Bhattacharyya, Elena Grigorescu, Madhav Jha, Kyomin Jung, Sofya Raskhodnikova, and David P. Woodruff. Lower bounds for local monotonicity reconstruction from transitive-closure spanners. In Maria J. Serna, Ronen Shaltiel, Klaus Jansen, and José D. P. Rolim, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, 13th International Workshop, APPROX 2010, and 14th International Workshop, RANDOM 2010, Barcelona, Spain, September 1-3, 2010. Proceedings*, Lecture Notes in Computer Science, pages 448–461. Springer, 2010.
- [BKKM23] Mark Braverman, Subhash Khot, Guy Kindler, and Dor Minzer. Improved monotonicity testers via hypercube embeddings. In Yael Tauman Kalai, editor, *14th Innovations in Theoretical Computer Science Conference, ITCS 2023, MIT, Cambridge, Massachusetts, USA, January 10-13, 2023*, LIPIcs, pages 25:1–25:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.
- [BKR24] Hadley Black, Iden Kalemaj, and Sofya Raskhodnikova. Isoperimetric inequalities for real-valued functions with applications to monotonicity testing. *Random Struct. Algorithms*, 65(1):191–219, 2024.
- [BKS23a] Sayan Bhattacharya, Peter Kiss, and Thatchaphol Saranurak. Dynamic $(1 + \varepsilon)$ -approximate matching size in truly sublinear update time. In *64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*, pages 1563–1588. IEEE, 2023.
- [BKS23b] Sayan Bhattacharya, Peter Kiss, and Thatchaphol Saranurak. Sublinear algorithms for $(1.5 + \varepsilon)$ -approximate matching. In Barna Saha and Rocco A. Servedio, editors, *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, pages 254–266. ACM, 2023.
- [BKSW24] Sayan Bhattacharya, Peter Kiss, Thatchaphol Saranurak, and David Wajc. Dynamic matching with better-than-2 approximation in polylogarithmic update time. *J. ACM*, 71(5):33:1–33:32, 2024.
- [BRR23a] Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinfeld. Local computation algorithms for maximum matching: New lower bounds. In *64th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2023, Santa Cruz, CA, USA, November 6-9, 2023*, pages 2322–2335. IEEE, 2023.

- [BRR23b] Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinstein. Sublinear time algorithms and complexity of approximate maximum matching. In Barna Saha and Rocco A. Servedio, editors, *Proceedings of the 55th Annual ACM Symposium on Theory of Computing, STOC 2023, Orlando, FL, USA, June 20-23, 2023*, pages 267–280. ACM, 2023.
- [BRR24] Soheil Behnezhad, Mohammad Roghani, and Aviad Rubinstein. Approximating maximum matching requires almost quadratic time. In Bojan Mohar, Igor Shinkar, and Ryan O’Donnell, editors, *Proceedings of the 56th Annual ACM Symposium on Theory of Computing, STOC 2024, Vancouver, BC, Canada, June 24-28, 2024*, pages 444–454. ACM, 2024.
- [BRRS23] Soheil Behnezhad, Mohammad Roghani, Aviad Rubinstein, and Amin Saberi. Beating greedy matching in sublinear time. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 3900–3945. SIAM, 2023.
- [BRRS24] Soheil Behnezhad, Mohammad Roghani, Aviad Rubinstein, and Amin Saberi. Sublinear algorithms for TSP via path covers. In Karl Bringmann, Martin Grohe, Gabriele Puppis, and Ola Svensson, editors, *51st International Colloquium on Automata, Languages, and Programming, ICALP 2024, Tallinn, Estonia, July 8-12, 2024*, LIPIcs, pages 19:1–19:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2024.
- [BRY14] Piotr Berman, Sofya Raskhodnikova, and Grigory Yaroslavtsev. L_p -testing. In David B. Shmoys, editor, *Symposium on Theory of Computing, STOC 2014, New York, NY, USA, May 31 - June 03, 2014*, pages 164–173. ACM, 2014.
- [CDH⁺25] Xi Chen, Anindya De, Yizhi Huang, Yuhao Li, Shivam Nadimpalli, Rocco A. Servedio, and Tianqi Yang. Relative-error monotonicity testing. In Yossi Azar and Debmalaya Panigrahi, editors, *Proceedings of the 2025 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2025, New Orleans, LA, USA, January 12-15, 2025*, pages 373–402. SIAM, 2025.
- [CDL⁺24] Xi Chen, Anindya De, Yuhao Li, Shivam Nadimpalli, and Rocco A. Servedio. Mildly exponential lower bounds on tolerant testers for monotonicity, unateness, and juntas. In David P. Woodruff, editor, *Proceedings of the 2024 ACM-SIAM Symposium on Discrete Algorithms, SODA 2024, Alexandria, VA, USA, January 7-10, 2024*, pages 4321–4337. SIAM, 2024.
- [CDST15] Xi Chen, Anindya De, Rocco A. Servedio, and Li-Yang Tan. Boolean function monotonicity testing requires (almost) $n^{1/2}$ non-adaptive queries. In Rocco A. Servedio and Ronitt Rubinfeld, editors, *Proceedings of the Forty-Seventh Annual ACM on Symposium on Theory of Computing, STOC 2015, Portland, OR, USA, June 14-17, 2015*, pages 519–528. ACM, 2015.
- [CFPS20] Artur Czumaj, Hendrik Fichtenberger, Pan Peng, and Christian Sohler. Testable properties in general graphs and random order streaming. In Jaroslav Byrka and Raghu Meka, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2020, Virtual Conference, August 17-19, 2020*, LIPIcs, pages 16:1–16:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.

- [CKK20] Yu Chen, Sampath Kannan, and Sanjeev Khanna. Sublinear algorithms and lower bounds for metric TSP cost estimation. In Artur Czumaj, Anuj Dawar, and Emanuela Merelli, editors, *47th International Colloquium on Automata, Languages, and Programming, ICALP 2020, Saarbrücken, Germany (Virtual Conference), July 8-11, 2020*, LIPIcs, pages 30:1–30:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2020.
- [CKP⁺21] Lijie Chen, Gillat Kol, Dmitry Paramonov, Raghuvansh Saxena, Zhao Song, and Huacheng Yu. Almost optimal super-constant-pass streaming lower bounds for reachability. *Electron. Colloquium Comput. Complex.*, TR21, 2021.
- [CKT23a] Yu Chen, Sanjeev Khanna, and Zihan Tan. Query complexity of the metric steiner tree problem. In Nikhil Bansal and Viswanath Nagarajan, editors, *Proceedings of the 2023 ACM-SIAM Symposium on Discrete Algorithms, SODA 2023, Florence, Italy, January 22-25, 2023*, pages 4893–4935. SIAM, 2023.
- [CKT23b] Yu Chen, Sanjeev Khanna, and Zihan Tan. Sublinear algorithms and lower bounds for estimating MST and TSP cost in general metrics. In Kousha Etessami, Uriel Feige, and Gabriele Puppis, editors, *50th International Colloquium on Automata, Languages, and Programming, ICALP 2023, Paderborn, Germany, July 10-14, 2023*, LIPIcs, pages 37:1–37:16. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.
- [CLM⁺20] Artur Czumaj, Jakub Łacki, Aleksander Mądry, Slobodan Mitrović, Krzysztof Onak, and Piotr Sankowski. Round compression for parallel matching algorithms. *SIAM Journal on Computing*, 49(5):STOC18–1–STOC18–44, 2020.
- [CS13] Deeparnab Chakrabarty and C. Seshadhri. Optimal bounds for monotonicity and Lipschitz testing over hypercubes and hypergrids. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *Symposium on Theory of Computing Conference, STOC’13, Palo Alto, CA, USA, June 1-4, 2013*, pages 419–428. ACM, 2013.
- [CS14] Deeparnab Chakrabarty and C. Seshadhri. An optimal lower bound for monotonicity testing over hypergrids. *Theory Comput.*, 10:453–464, 2014.
- [CS16] Deeparnab Chakrabarty and C. Seshadhri. An $o(n)$ monotonicity tester for boolean functions over the hypercube. *SIAM J. Comput.*, 45(2):461–472, 2016.
- [CS19] Deeparnab Chakrabarty and C. Seshadhri. Adaptive boolean monotonicity testing in total influence time. In Avrim Blum, editor, *10th Innovations in Theoretical Computer Science Conference, ITCS 2019, San Diego, California, USA, January 10-12, 2019*, LIPIcs, pages 20:1–20:7. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2019.
- [CS25] Deeparnab Chakrabarty and C. Seshadhri. Directed hypercube routing, a generalized lehmman-ron theorem, and monotonicity testing. In Raghu Meka, editor, *16th Innovations in Theoretical Computer Science Conference, ITCS 2025, Columbia University, New York, NY, USA, January 7-10, 2025*, LIPIcs, pages 34:1–34:15. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.
- [CST14] Xi Chen, Rocco A. Servedio, and Li-Yang Tan. New algorithms and lower bounds for monotonicity testing. In *55th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2014, Philadelphia, PA, USA, October 18-21, 2014*, pages 286–295. IEEE Computer Society, 2014.

- [CWX17] Xi Chen, Erik Waingarten, and Jinyu Xie. Beyond talagrand functions: new lower bounds for testing monotonicity and unateness. In Hamed Hatami, Pierre McKenzie, and Valerie King, editors, *Proceedings of the 49th Annual ACM SIGACT Symposium on Theory of Computing, STOC 2017, Montreal, QC, Canada, June 19-23, 2017*, pages 523–536. ACM, 2017.
- [DGL⁺99] Yevgeniy Dodis, Oded Goldreich, Eric Lehman, Sofya Raskhodnikova, Dana Ron, and Alex Samorodnitsky. Improved testing algorithms for monotonicity. In Dorit S. Hochbaum, Klaus Jansen, José D. P. Rolim, and Alistair Sinclair, editors, *Randomization, Approximation, and Combinatorial Algorithms and Techniques, Third International Workshop on Randomization and Approximation Techniques in Computer Science, and Second International Workshop on Approximation Algorithms for Combinatorial Optimization Problems RANDOM-APPROX’99, Berkeley, CA, USA, August 8-11, 1999, Proceedings*, Lecture Notes in Computer Science, pages 97–108. Springer, 1999.
- [Edm65] Jack Edmonds. Maximum matching and a polyhedron with 0, 1-vertices. *Journal of Research of the National Bureau of Standards B*, 69:125–130, 1965.
- [EKK⁺00] Funda Ergün, Sampath Kannan, Ravi Kumar, Ronitt Rubinfeld, and Mahesh Viswanathan. Spot-checkers. *J. Comput. Syst. Sci.*, 60(3):717–751, 2000.
- [Fis04] Eldar Fischer. On the strength of comparisons in property testing. *Inf. Comput.*, 189(1):107–116, 2004.
- [FJ25] Yumou Fei and Renato Ferreira Pinto Jr. On the spectral expansion of monotone subsets of the hypercube. In Alina Ene and Eshan Chattopadhyay, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2025, Berkeley, CA, USA, August 11-13, 2025*, LIPIcs, pages 42:1–42:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.
- [FLN⁺02] Eldar Fischer, Eric Lehman, Ilan Newman, Sofya Raskhodnikova, Ronitt Rubinfeld, and Alex Samorodnitsky. Monotonicity testing over general poset domains. In John H. Reif, editor, *Proceedings on 34th Annual ACM Symposium on Theory of Computing, May 19-21, 2002, Montréal, Québec, Canada*, pages 474–483. ACM, 2002.
- [GGL⁺00] Oded Goldreich, Shafi Goldwasser, Eric Lehman, Dana Ron, and Alex Samorodnitsky. Testing monotonicity. *Comb.*, 20(3):301–337, 2000.
- [JLS19] Arun Jambulapati, Yang P. Liu, and Aaron Sidford. Parallel reachability in almost linear work and square root depth. In David Zuckerman, editor, *60th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2019, Baltimore, Maryland, USA, November 9-12, 2019*, pages 1664–1686. IEEE Computer Society, 2019.
- [Jr.23] Renato Ferreira Pinto Jr. Directed Poincaré inequalities and L^1 monotonicity testing of Lipschitz functions. In Nicole Megow and Adam D. Smith, editors, *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques, APPROX/RANDOM 2023, Atlanta, Georgia, USA, September 11-13, 2023*, LIPIcs, pages 61:1–61:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2023.

- [Jr.24] Renato Ferreira Pinto Jr. Directed isoperimetry and monotonicity testing: A dynamical approach. In *65th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2024, Chicago, IL, USA, October 27-30, 2024*, pages 2295–2305. IEEE, 2024.
- [KKS14] Michael Kapralov, Sanjeev Khanna, and Madhu Sudan. Approximating matching size from random streams. In Chandra Chekuri, editor, *Proceedings of the Twenty-Fifth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2014, Portland, Oregon, USA, January 5-7, 2014*, pages 734–751. SIAM, 2014.
- [KMNFT20] Michael Kapralov, Slobodan Mitrović, Ashkan Norouzi-Fard, and Jakab Tardos. Space efficient approximation to maximum matching size from uniform edge samples. In *Proceedings of the Thirty-First Annual ACM-SIAM Symposium on Discrete Algorithms, SODA '20*, page 1753–1772, USA, 2020. Society for Industrial and Applied Mathematics.
- [KMNT20] Michael Kapralov, Slobodan Mitrovic, Ashkan Norouzi-Fard, and Jakab Tardos. Space efficient approximation to maximum matching size from uniform edge samples. In Shuchi Chawla, editor, *Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms, SODA 2020, Salt Lake City, UT, USA, January 5-8, 2020*, pages 1753–1772. SIAM, 2020.
- [KMS18] Subhash Khot, Dor Minzer, and Muli Safra. On monotonicity testing and boolean isoperimetric-type theorems. *SIAM J. Comput.*, 47(6):2238–2276, 2018.
- [Liu24] Yang P. Liu. On approximate fully-dynamic matching and online matrix-vector multiplication. In *65th IEEE Annual Symposium on Foundations of Computer Science, FOCS 2024, Chicago, IL, USA, October 27-30, 2024*, pages 228–243. IEEE, 2024.
- [LRV22] Jane Lange, Ronitt Rubinfeld, and Arsen Vasilyan. Properly learning monotone functions via local correction. In *63rd IEEE Annual Symposium on Foundations of Computer Science, FOCS 2022, Denver, CO, USA, October 31 - November 3, 2022*, pages 75–86. IEEE, 2022.
- [LRY17] Reut Levi, Ronitt Rubinfeld, and Anak Yodpinyanee. Local computation algorithms for graphs of non-constant degrees. *Algorithmica*, 77(4):971–994, 2017.
- [MMPS17] Morteza Monemizadeh, S. Muthukrishnan, Pan Peng, and Christian Sohler. Testable bounded degree graph properties are random order streamable. In Ioannis Chatzigiannakis, Piotr Indyk, Fabian Kuhn, and Anca Muscholl, editors, *44th International Colloquium on Automata, Languages, and Programming, ICALP 2017, Warsaw, Poland, July 10-14, 2017*, LIPIcs, pages 131:1–131:14. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2017.
- [MRT25] Sepideh Mahabadi, Mohammad Roghani, and Jakub Tarnawski. A 0.51-approximation of maximum matching in sublinear $n^{1.5}$ time. In Keren Censor-Hillel, Fabrizio Grandoni, Joël Ouaknine, and Gabriele Puppis, editors, *52nd International Colloquium on Automata, Languages, and Programming (ICALP 2025)*, volume 334 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 116:1–116:17, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.

- [MRTV25] Sepideh Mahabadi, Mohammad Roghani, Jakub Tarnawski, and Ali Vakilian. Sublinear metric steiner tree via improved bounds for set cover. In Raghu Meka, editor, *16th Innovations in Theoretical Computer Science Conference, ITCS 2025, Columbia University, New York, NY, USA, January 7-10, 2025*, LIPIcs, pages 74:1–74:24. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.
- [MRTV26] Sepideh Mahabadi, Mohammad Roghani, Jakub Tarnawski, and Ali Vakilian. Sublinear metric steiner forest via maximal independent set. In Kasper Green Larsen and Barna Saha, editors, *Proceedings of the 2026 Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2026, Vancouver, BC, Canada, January 11-14, 2026*, pages 5941–5959. SIAM, 2026.
- [NO08] Huy N. Nguyen and Krzysztof Onak. Constant-time approximation algorithms via local improvements. In *49th Annual IEEE Symposium on Foundations of Computer Science, FOCS 2008, Philadelphia, PA, USA, October 25-28, 2008*, pages 327–336. IEEE Computer Society, 2008.
- [OR10] Krzysztof Onak and Ronitt Rubinfeld. Maintaining a large matching and a small vertex cover. In Leonard J. Schulman, editor, *Proceedings of the 42nd ACM Symposium on Theory of Computing, STOC 2010, Cambridge, Massachusetts, USA, 5-8 June 2010*, pages 457–464. ACM, 2010.
- [ORRR12] Krzysztof Onak, Dana Ron, Michal Rosen, and Ronitt Rubinfeld. A near-optimal sublinear-time algorithm for approximating the minimum vertex cover size. In Yuval Rabani, editor, *Proceedings of the Twenty-Third Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2012, Kyoto, Japan, January 17-19, 2012*, pages 1123–1131. SIAM, 2012.
- [PR07] Michal Parnas and Dana Ron. Approximating the minimum vertex cover in sublinear time and a connection to distributed algorithms. *Theor. Comput. Sci.*, 381(1-3):183–196, 2007.
- [PRR06] Michal Parnas, Dana Ron, and Ronitt Rubinfeld. Tolerant property testing and distance approximation. *J. Comput. Syst. Sci.*, 72(6):1012–1042, 2006.
- [PRW22] Ramesh Krishnan S. Pallavoor, Sofya Raskhodnikova, and Erik Waingarten. Approximating the distance to monotonicity of boolean functions. *Random Struct. Algorithms*, 60(2):233–260, 2022.
- [PS18] Pan Peng and Christian Sohler. Estimating graph parameters from random order streams. In Artur Czumaj, editor, *Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms, SODA 2018, New Orleans, LA, USA, January 7-10, 2018*, pages 2449–2466. SIAM, 2018.
- [SS10] Michael E. Saks and C. Seshadhri. Local monotonicity reconstruction. *SIAM J. Comput.*, 39(7):2897–2926, 2010.
- [Yos26] Yuichi Yoshida. Testing monotonicity of real-valued functions on dags. *CoRR*, abs/2602.15341, 2026.
- [YYI09] Yuichi Yoshida, Masaki Yamamoto, and Hiro Ito. An improved constant-time approximation algorithm for maximum matchings. In Michael Mitzenmacher, editor,

Proceedings of the 41st Annual ACM Symposium on Theory of Computing, STOC 2009, Bethesda, MD, USA, May 31 - June 2, 2009, pages 225–234. ACM, 2009.